

Asynchronous API

Below, different variations of asynchronous APIs built on top of C-style callbacks with examples of doing 2 GET requests - both sequentially and concurrently are showcased.

NO threads are involved intentionally, to disconnect any associations of coroutines or fibers with multithreading. Some parts are deliberately simple, while still presenting as much details as possible.

Jump to examples for [blocking requests](#), [callbacks](#), [tasks .then\(\)](#), [std::future \(polling\)](#), [coroutines](#), [fibers](#), [senders](#).

Work In Progress.

View: [HTML](#), [PDF](#), [source code](#).

Contents

1	#introduction	2
2	#setup with CMake + libcurl	5
3	#building blocking API	6
3.1	#run simple http server for tests	8
4	#building C-style callbacks API	8
4.1	#thoughts on the design	8
4.2	#note on C-style API (vs C++)	10
4.3	#implementing with libcurl multi	11
5	#building C++20 coroutines API	16
5.1	#C++ coroutines, basic task type	17
5.2	#C++ coroutines, basic await	22
5.3	#C++ coroutines, await callback with a crash	24
5.4	#C++ coroutines, await callback	27
5.5	#coroutine task that holds a return value	29
5.6	#awaiting coroutine task	32
5.7	#waiting for multiple coroutines	37
5.8	#waiting for multiple CURL requests with tasks	40

5.9	#waiting for multiple CURL request with custom awaitable . . .	40
6	#building Fibers API	44
6.1	#basic Fiber	46
6.2	#switching between Fibers	49
6.3	#tasks for Fibers	51
6.4	#cancellable Fiber task with a generic callback	52
6.5	#basic FiberPool	57
6.6	#FiberTask	59
6.7	#waiting for a CURL request with a Fiber	66
6.8	#waiting for multiple a FiberTasks	71
7	#building std::future API	72
8	#building task API with .then() support	73
8.1	#wrapping CURL get into a Task	82
8.2	#waiting for multiple Task requests	83
9	#building C++26 senders	86
9.1	#sender for a CURL get	89
9.2	#senders basics: implementing then() and sync_wait()	95
9.3	#senders basics: implementing when_all()	103
9.4	#senders basics: implementing sequence()	111
9.5	#senders basics: CURL get	113
10	#reactive streams	113
11	#SAMPLES	113
11.1	#synchronous requests	113
11.2	#requests with callbacks	114
11.3	#requests with coroutines	117
11.4	#requests with fibers	118
11.5	#polling requests with std::futures	119
11.6	#requests with Tasks .then()	122
11.7	#requests with senders/std::execution	123

1 #introduction

We start with a simple C-style API on top of [libcurl C API](#) and have a code that may look like this:

```

1 // our CURL API
2 std::string CURL_get(const std::string& url);
3
4 int main()
```

```

5 {
6     const std::string r1 = CURL_get("localhost:5001/file1.txt");
7     const std::string r2 = CURL_get("localhost:5001/file2.txt");
8     std::println("{} ", r1);
9     std::println("{} ", r2);
10 }

```

The code above performs two GET requests sequentially. Everything executes synchronously.

Next, let's have a simple C-style callbacks API (**not** the C++ one, see [the note](#)), to run requests concurrently:

```

1 // libcurl bookkeeping
2 using CURL_Async = void*;
3 CURL_Async CURL_async_create();
4 void CURL_async_destroy(CURL_Async curl_async);
5 void CURL_async_tick(CURL_Async curl_async);
6
7 // main async callback API
8 void CURL_async_get(CURL_Async curl_async
9     , const std::string& url
10    , void* user_data
11    , void (*callback)(void* user_data, std::string response));

```

Doing 2 GET requests is more involved now:

```

1 int main()
2 {
3     struct State
4     {
5         int count = 0;
6         std::string r1;
7         std::string r2;
8     };
9
10    CURL_Async curl_async = CURL_async_create();
11    State state;
12    CURL_async_get(curl_async, "localhost:5001/file1.txt", &state
13        , [] (void* user_data, std::string response)
14        {
15            State& state = *static_cast<State*>(user_data);
16            state.count += 1;
17            state.r1 = std::move(response);
18        });
19    CURL_async_get(curl_async, "localhost:5001/file2.txt", &state
20        , [] (void* user_data, std::string response)

```

```

21     {
22         State& state = *static_cast<State*>(user_data);
23         state.count += 1;
24         state.r2 = std::move(response);
25     });
26     while (state.count != 2) // wait for 2 requests to finish
27     {
28         CURL_async_tick(curl_async);
29     }
30     CURL_async_destroy(curl_async);
31     std::println("{} ", state.r1);
32     std::println("{} ", state.r2);
33 }

```

There is a need to have a `State` for bookkeeping, pass it as a `void*` user data to access later and, finally, run an event loop to give libcurl a chance to process requests. Note, however, requests execute concurrently now, as in - 2 requests are active at the same time.

After this, let's build other variations of asynchronous API on top of C-style callbacks above:

```

1  std::string CURL_get(const std::string& url);
2
3  void CURL_async_get(CURL_Async curl_async
4      , const std::string& url
5      , void* user_data
6      , void (*callback)(void* user_data, std::string response));
7
8  Co_CurlAsync CURL_await_get(
9      CURL_Async curl_async, const std::string& url);
10 Co_Task<std::string> CURL_coro_get(
11     CURL_Async curl_async, std::string url);
12
13 std::string CURL_fiber_get(
14     CURL_Async curl_async, const std::string& url);
15 FiberTask<std::string> CURL_fiber_get(
16     CURL_Async curl_async
17     , const std::string& url
18     , FiberTaskScheduler& fiber_scheduler);
19
20 std::future<std::string> CURL_future_get(
21     CURL_Async curl_async, const std::string& url);
22 Task<std::string> CURL_task_get(
23     CURL_Async curl_async, const std::string& url);
24

```

```

25  CURL_Get_Sender CURL_sender_get(
26      CURL_Async curl_async, const std::string& url);

```

But before that, lets wrap [libcurl C API](#) for our needs.

2 #setup with CMake + libcurl

[source code](#).

For [vcpkg](#), there is an extensive [documentation](#) available. In short:

```

1  git clone https://github.com/microsoft/vcpkg
2  cd vcpkg
3  bootstrap-vcpkg.bat

```

Assuming vcpkg is installed at K:\vcpkg, we also set VCPKG_ROOT and add this path to PATH environment variable so build scripts can use VCPKG_ROOT and vcpkg without a need to know the exact location.

```

1  set VCPKG_ROOT=K:\vcpkg
2  set PATH=%VCPKG_ROOT%;%PATH%

```

For the project (lets say in a K:\async_api folder), vcpkg [manifest mode](#) is used. Together with curl setup, all required steps are

```

1  cd K:\async_api
2  vcpkg new --application
3  vcpkg add port curl

```

Note that to find exact curl package name, vcpkg `search curl` was used which prints:

```
curl 8.13.0#1 A library for transferring data with URLs
```

CMakeLists.txt now looks like this:

```

1  cmake_minimum_required(VERSION 3.24 FATAL_ERROR)
2  project(async_api LANGUAGES CXX)
3
4  add_executable(CH00_cmake main.cc)
5  target_compile_features(CH00_cmake PUBLIC cxx_std_23)
6  find_package(CURL REQUIRED)
7  target_link_libraries(CH00_cmake PRIVATE CURL::libcurl)

```

`find_package(CURL REQUIRED)` syntax together with `CURL::libcurl` target name is found from the output log of `vcpkg install curl` (or during CMake configuration run) which prints:

```

1  curl is compatible with built-in CMake targets:
2
3      find_package(CURL REQUIRED)
4      target_link_libraries(main PRIVATE CURL::libcurl)

```

To test that everything compiles and links, save main.cc:

```

1  #include <curl/curl.h>
2
3  int main()
4  {
5      CURL* curl = curl_easy_init();
6      assert(curl);
7      curl_easy_cleanup(curl);
8  }

```

Finally, to invoke CMake configure, build and run (with vcpkg):

```

1  cd K:\async_api
2  cmake -S . -B __build ^
3      -DCMAKE_TOOLCHAIN_FILE=%VCPKG_ROOT%\scripts\buildsystems\vcpkg.cmake
4  cmake --build __build --config Debug
5  :: run a test
6  .\__build\Debug\CH00_cmake.exe

```

This assumes `cmake.exe` is in your PATH, see `build.cmd`.

3 #building blocking API

[source code](#), [sample app](#).

Blocking, synchronous API for a GET request is straightforward. We go with a function that looks like this:

```

1  std::string CURL_get(const std::string& url);

```

libcurl comes with two different APIs, “easy” and “multi”. Lets use an easy interface; libcurl examples available online, including official [simple.c example](#) for a start.

Everything together leads to the implementation below, where `curl_easy_perform()` call is the main one that blocks the execution until request completes; once complete, we can return results:

```

1  #include <string>
2
3  #include <curl/curl.h>
4
5  #if defined(NDEBUG)

```

```

6  # undef NDEBUG
7  #endif
8  #include <cassert>
9
10 static size_t CURL_OnWriteCallback(
11     void* ptr, size_t size, size_t nmemb, void* data)
12 {
13     std::string& response = *static_cast<std::string*>(data);
14     response.append(static_cast<const char*>(ptr), size * nmemb);
15     return (size * nmemb);
16 }
17
18 std::string CURL_get(const std::string& url)
19 {
20     CURL* curl = curl_easy_init();
21     assert(curl);
22
23     CURLcode status = curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
24     assert(status == CURLE_OK);
25     status = curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);
26     assert(status == CURLE_OK);
27
28     std::string response;
29     status = curl_easy_setopt(curl
30         , CURLOPT_WRITEFUNCTION, CURL_OnWriteCallback);
31     assert(status == CURLE_OK);
32     status = curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
33     assert(status == CURLE_OK);
34
35     status = curl_easy_perform(curl);
36     assert(status == CURLE_OK);
37
38     long response_code = -1;
39     status = curl_easy_getinfo(curl, CURLINFO_RESPONSE_CODE, &response_code);
40     assert(status == CURLE_OK);
41     assert(response_code == 200L);
42
43     curl_easy_cleanup(curl);
44     return response;
45 }

```

Note on error handling: for now, we crash on any unexpected error - as in “crash the whole application”. `assert()` is enabled always **intentionally** to simplify both, the sample code **and** debugging:

```

1 // after all includes, main.cc
2 #if defined(NDEBUG)
3 #   undef NDEBUG
4 #endif
5 #include <cassert>

```

This is “bad” for generic, low-level library API/code, but could be fine sometimes. We’ll [discuss error handling later](#).

To see the code in action, lets run our program:

```

1 #include <print>
2
3 int main()
4 {
5     const std::string r = CURL_get("localhost:5001/file1.txt");
6     std::println("{} ", r);
7 }

```

that.. should crash since we don’t have local HTTP server running to serve localhost:5001/file1.txt. See the [next section on how to make it happen](#).

Once done, we should see the sample file1.txt content in the console output:

Fox

3.1 [#run simple http server for tests](#)

[source code](#).

To run sample code, lets use Python to have simple HTTP server that hosts files in the current directory, see `serve.cmd`:

```

1 python -m http.server 5001

```

Given the directory that has [file1.txt](#) and [file2.txt](#) `CURL_get("localhost:5001/file1.txt")` should work and return the content of the file, see [blocking libcurl section](#).

4 [#building C-style callbacks API](#)

[source code](#), [sample app](#).

4.1 [#thoughts on the design](#)

Now, lets imagine simplest possible asynchronous API. The difference to [blocking API](#) is that we ask the system to start a GET request and the response could arrive some time later. The system invokes a user-provided `callback` to notify us once everything is done:

```

1 void CURL_async_get(const std::string& url
2     , void (*callback)(std::string));

```

we could use it like this:

```

1 // start a request:
2 CURL_async_get("localhost:5001/file2.txt"
3     , [](std::string response)
4 {
5     // probably, some time later:
6     std::println("got response: {}", response);
7 });

```

There are multiple issues with the design above:

1. Where is the “system” that starts the request? It could be implicit, hidden global singleton, but we can also ask a user to explicitly create and pass it around.
2. The callback accepts only **response**, there is no way for a user to access other data, without resorting to global singletons again. When starting a request, user should be able to provide opaque pointer to some data that system does not touch and simply gives it back in callback.
3. When and from where the “system” invokes a **callback**? There are multiple answers, but we go with user-controlled event loop that drives everything.

To solve first issue, lets have explicit API to create and destroy the system:

```

1 using CURL_Async = void*; // system's state
2
3 CURL_Async CURL_async_create();
4 void CURL_async_destroy(CURL_Async curl_async);

```

where **CURL_Async** is the system itself, since user does not care what’s that exactly, it’s hidden under **void***. User creates the system, uses it and, once not needed, destroys to clean up resources, if any.

To drive a system with event loop, user must call the next API:

```

1 void CURL_async_tick(CURL_Async curl_async);

```

This is the chance for a system to actually do some work over time **and** invoke user-provided callbacks, if needed.

Lastly, to give a user some control over data in the callback, we pass opaque **void*** pointer around:

```

1 // main async callback API
2 void CURL_async_get(CURL_Async curl_async
3     , const std::string& url

```

```

4     , void* user_data
5     , void (*callback)(void* user_data, std::string response));

```

`user_data` could be anything, system gives it back when invoking `callback`. This is user responsibility to ensure that pointer is valid all the time while request is in progress.

There are more nuances, like how frequently/when `CURL_async_tick()` should be invoked by a user; but all this is left out of the scope.

Overall, everything included, we need to implement the next API, see [below](#):

```

1 // libcurl bookkeeping
2 using CURL_Async = void*;
3 CURL_Async CURL_async_create();
4 void CURL_async_destroy(CURL_Async curl_async);
5 void CURL_async_tick(CURL_Async curl_async);
6
7 // main async callback API
8 void CURL_async_get(CURL_Async curl_async
9     , const std::string& url
10    , void* user_data
11    , void (*callback)(void* user_data, std::string response));

```

4.2 #note on C-style API (vs C++)

For [C-style API above](#), with C++, “the system” could be a class, callback could be `std::function<>` to accept anything, generally making it less verbose, having something like this:

```

1 // the API:
2 class CURL_Async
3 {
4 public:
5     void get(const std::string& url, std::function<void (std::string)>);
6     void tick();
7 };
8
9 // the use:
10 CURL_Async curl;
11 curl.get("localhost:5001/file1.txt", [](std::string r)
12 {
13     std::println("{} ", r);
14 });
15 curl.tick(); // etc

```

However, C-style API we have, is a de-facto standard, familiar and recognized for asynchronous APIs with callbacks (citation needed).

The rest of asynchronous APIs implementations below are built on top of C-style callback API, as a basic building block to cover similar callbacks-based APIs.

4.3 #implementing with libcurl multi

[source code](#), [sample app](#).

For our callbacks [API](#):

```
1 using CURL_Async = void*;
2 CURL_Async CURL_async_create();
3 void CURL_async_destroy(CURL_Async curl_async);
4 void CURL_async_tick(CURL_Async curl_async);
5 void CURL_async_get(CURL_Async curl_async
6     , const std::string& url
7     , void* user_data
8     , void (*callback)(void* user_data, std::string response));
```

internally, lets have CURL_AsyncScheduler class to handle adding requests, updating/ticking libcurl event loop and, in general, to represent our whole CURL_Async system state:

```
1 struct CURL_AsyncScheduler
2 {
3     CURL_AsyncScheduler();
4     ~CURL_AsyncScheduler();
5     // no copy, no move
6     CURL_AsyncScheduler(const CURL_AsyncScheduler&) = delete;
7
8     using Callback = std::function<void (CURL* curl_easy)>;
9
10    void tick();
11    void add_request(CURL* curl_easy, Callback on_finish);
12
13    // our state
14    CURLM* _multi_curl = nullptr;
15    std::unordered_map<CURL*, Callback> _curl_to_callback;
16 };
```

This is what we'll return to a user as CURL_Async pointer. Lets do it:

```
1 CURL_Async CURL_async_create()
2 {
3     CURL_AsyncScheduler* scheduler = new(std::nothrow) CURL_AsyncScheduler();
4     assert(scheduler);
5     return scheduler;
6 }
7
```

```

8 void CURL_async_destroy(CURL_Async curl_async)
9 {
10     assert(curl_async);
11     CURL_AsyncScheduler* scheduler =
12         static_cast<CURL_AsyncScheduler*>(curl_async);
13     delete scheduler;
14 }

```

Now, user just needs to pass CURL_Async handle around. Before implementing internals, lets have a helper function that gets actual CURL_AsyncScheduler instance from opaque handle:

```

1 CURL_AsyncScheduler& CURL_scheduler(CURL_Async curl_async)
2 {
3     CURL_AsyncScheduler* scheduler =
4         static_cast<CURL_AsyncScheduler*>(curl_async);
5     assert(scheduler);
6     return *scheduler;
7 }

```

It's not exposed to the user in any way. Lets implement our main API in terms of our internal scheduler:

```

1 void CURL_async_get(CURL_Async curl_async
2     , const std::string& url
3     , void* user_data
4     , void (*callback)(void* user_data, std::string response))
5 {
6     // 1. setup curl easy handle
7     CURL* curl_easy = curl_easy_init();
8     assert(curl_easy);
9     CURLcode status = curl_easy_setopt(curl_easy, CURLOPT_URL, url.c_str());
10    assert(status == CURLE_OK);
11    status = curl_easy_setopt(curl_easy, CURLOPT_FOLLOWLOCATION, 1L);
12    assert(status == CURLE_OK);
13
14    // 2. write response data to separate std::string
15    std::string* state = new std::string{};
16    status = curl_easy_setopt(curl_easy
17        , CURLOPT_WRITEFUNCTION, CURL_OnWriteCallback);
18    assert(status == CURLE_OK);
19    status = curl_easy_setopt(curl_easy, CURLOPT_WRITEDATA, state);
20    assert(status == CURLE_OK);
21
22    // 3. associate with multi handle/event loop
23    CURL_scheduler(curl_async).add_request(curl_easy
24        , [state, user_data, callback](CURL* curl_easy)

```

```

25     {
26         long response_code = -1;
27         const CURLcode status = curl_easy_getinfo(curl_easy
28             , CURLINFO_RESPONSE_CODE, &response_code);
29         assert(status == CURLE_OK);
30         assert(response_code == 200L);
31         curl_easy_cleanup(curl_easy);
32         std::string data = std::move(*state);
33         delete state;
34         callback(user_data, std::move(data));
35     });
36 }

```

There are few moving parts and issues:

1. we create and setup curl easy handle in the same way as for blocking call;
2. we allocate separate `std::string` to write the response data to with the same `CURL_OnWriteCallback` callback as in the [blocking implementation](#);
3. finally, we associate the request with event loop/multi handle
4. new `std::string` will leak the memory if the request is not completed
5. overall, there are more hidden allocations from within `add_request()`:
 - a) `std::function<>` most likely allocates
 - b) `std::unordered_map` allocates

It could be done another way around, eliminating the need for separate `std::string` allocation and few more optimizations, mainly with the help of associating user data with curl easy handle/ [CURLOPT_PRIVATE](#). However, it's good enough for illustrative purposes.

After creation of curl easy handle, we associate it with curl multi handle:

```

1  CURL_AsyncScheduler::CURL_AsyncScheduler()
2  {
3      const CURLcode status = curl_global_init(CURL_GLOBAL_ALL);
4      assert(status == CURLE_OK);
5      _multi_curl = curl_multi_init();
6      assert(_multi_curl);
7  }
8
9  CURL_AsyncScheduler::~CURL_AsyncScheduler()
10 {
11     const CURLMcode status = curl_multi_cleanup(_multi_curl);
12     assert(status == CURLM_OK);
13     curl_global_cleanup();
14 }
15
16 void CURL_AsyncScheduler::add_request(CURL* curl_easy, Callback on_finish)

```

```

17 {
18     assert(on_finish);
19     assert(curl_easy);
20     assert(!_curl_to_callback.contains(curl_easy));
21
22     const CURLMcode status = curl_multi_add_handle(_multi_curl, curl_easy);
23     assert(status == CURLM_OK);
24     _curl_to_callback[curl_easy] = std::move(on_finish);
25 }

```

`_curl_to_callback` map is used to be able to retrieve callback later, given curl easy handle (CURL*).

Our user-exposed `CURL_async_tick()` API is implemented in terms of scheduler:

```

1 void CURL_async_tick(CURL_Async curl_async)
2 {
3     CURL_scheduler(curl_async).tick();
4 }
5
6 void CURL_AsyncScheduler::tick()
7 {
8     int running_handles = -1;
9     CURLMcode status = curl_multi_perform(_multi_curl, &running_handles);
10    assert(status == CURLM_OK);
11    int msgs_in_queue = 0;
12    while (CURLMsg* m = curl_multi_info_read(_multi_curl, &msgs_in_queue))
13    {
14        if (m->msg != CURLMSG_DONE)
15        {
16            continue;
17        }
18        CURL* curl_easy = m->easy_handle;
19        assert(curl_easy);
20        status = curl_multi_remove_handle(_multi_curl, curl_easy);
21        assert(status == CURLM_OK);
22        auto it = _curl_to_callback.find(curl_easy);
23        assert(it != _curl_to_callback.end());
24        Callback callback = std::move(it->second);
25        assert(callback);
26        (void)_curl_to_callback.erase(it);
27        callback(curl_easy);
28    }
29 }

```

The main part of event loop is the call to `curl_multi_perform()`. Once done we ask for easy handle requests that were completed, search for an associated

callback for each request and invoke it.

Note, there are no threads involved and it's possible to create many GET requests at once with multiple calls to `CURL_async_get()` - libcurl will manage them all together.

Again, it's user responsibility to drive libcurl with a periodic calls to `CURL_async_tick()`. Lets do single request with the API above:

```
1  #include <print>
2
3  int main()
4  {
5      struct State
6      {
7          std::string response;
8          bool done = false;
9      };
10     CURL_Async curl_async = CURL_async_create();
11     State state;
12     CURL_async_get(curl_async, "localhost:5001/file1.txt", &state
13         , [](void* user_data, std::string response)
14     {
15         State& state = *static_cast<State*>(user_data);
16         state.response = std::move(response);
17         state.done = true;
18     });
19     while (!state.done)
20     {
21         CURL_async_tick(curl_async);
22     }
23     CURL_async_destroy(curl_async);
24
25     std::println("async response: '{}'", state.response);
26 }
```

If [python HTTP server](#) is running, our program should print:

```
1  Fox
```

[#on error handling](#)
[#assume success always \(tooling\)](#)
[#implicit, return empty string](#)
[#status code, out parameter \(std::filesystem-style\)](#)
[#optional](#)
[#exceptions](#)
[#result/variant-like](#)
[#result/tuple-like](#)
[#result/specialized](#)
[#async with statefull/implicit callback \(state.on__X.subscribe/delegat](#)

5 [#building C++20 coroutines API](#)

[sample app.](#)

Coroutines materials:

- [How C++ coroutines work.](#)
- All of [Asymmetric Transfer](#), author of [cppcoro](#).

In short, we'd like to be able to write something like this:

```

1  const std::string response = co_await CURL_await_get(
2      curl_async, "localhost:5001/file1.txt");
3  // use `response` as a usual variable, no callbacks

```

There are several moving and a bit unrelative parts to have working coroutines code. First, coroutine function return type needs to be built, just to be able to write any/empty coroutine:

```

1  Co_Task coro_work()
2  {
3      co_return;
4  }

```

Next, there is a need to write coroutine awaitable to be able to `co_await` some work, specifically, GET request:

```

1  Co_Task coro_work(CURL_Async curl_async)
2  {
3      std::string response = co_await CURL_await_get(curl_async
4          , "localhost:5001/file1.txt");
5      co_return;
6  }

```

And, finally, there are some challenges to have a code that has several GET requests on the fly, with coroutines.

Lets start with basics.

5.1 #C++ coroutines, basic task type

[source code](#).

There is a trick to writing some basic C++20 coroutines code - **listen to the compiler**. Lets see what it takes to make the next code “work”:

```
1 Co_Task coro_work()
2 {
3     co_return;
4 }
```

Co_Task is a class, lets have empty one and try to compile:

```
1 struct Co_Task {};
2
3 Co_Task coro_work()
4 {
5     co_return;
6 }
```

MSVC complains:

```
1 main.cc(164,5): error C3774: cannot find 'std::coroutine_traits':
2             Please include <coroutine> header
```

after including <coroutine> header:

```
1 main.cc(166,5): error C2039: 'promise_type': is not a member of
2             'std::coroutine_traits<Co_Task>'
```

Lets add empty promise_type class inside Co_Task:

```
1 #include <coroutine>
2
3 struct Co_Task
4 {
5     struct promise_type {};
6 };
7
8 Co_Task coro_work()
9 {
10     co_return;
11 }
```

MSVC complains:

```
1 main.cc(170,1): error C3789: this function cannot be a coroutine:
2             'Co_Task::promise_type' does not declare the member
3             'get_return_object()'
4 main.cc(170,1): error C3789: this function cannot be a coroutine:
5             'Co_Task::promise_type' does not declare the member
6             'initial_suspend()'
7 main.cc(170,1): error C3789: this function cannot be a coroutine:
8             'Co_Task::promise_type' does not declare the member
9             'final_suspend()'
```

Ah, so `promise_type` should have `get_return_object()`, `initial_suspend()` and `final_suspend()` member functions. Return types are unclear, unfortunately. To speed-up things, we know that `get_return_object()` should return `Co_Task`. For `initial_suspend()` and `final_suspend()` we'll go with `std::suspend_always` awaitables for now. That gives:

```
1 #include <coroutine>
2
3 struct Co_Task
4 {
5     struct promise_type
6     {
7         Co_Task get_return_object()          { return {}; }
8         std::suspend_always initial_suspend() { return {}; }
9         std::suspend_always final_suspend()   { return {}; }
10    };
11 };
12
13 Co_Task coro_work()
14 {
15     co_return;
16 }
```

MSVC complains:

```
1 main.cc(164,12): error C3781: Co_Task::promise_type: a coroutine's
2                 promise must declare either
3                 'return_value' or 'return_void'
4 main.cc(176,1):  error C2039: 'unhandled_exception': is not a member
5                 of 'Co_Task::promise_type'
```

Since our `coro_work()` coroutine has just `co_return`, we should provide `return_void()` member function. With `unhandled_exception()`, we have:

```
1 struct promise_type
2 {
```

```

3     Co_Task get_return_object()          { return {}; }
4     std::suspend_always initial_suspend() { return {}; }
5     std::suspend_always final_suspend()   { return {}; }
6     void return_void()                   {}
7     void unhandled_exception()           {}
8 };

```

MSVC complains:

```

1 main.cc(168,29): error C5231: the expression
2             'co_await promise.final_suspend()' must be non-throwing

```

OK, makes sense. Finally,

```

1 #include <coroutine>
2
3 struct Co_Task
4 {
5     struct promise_type
6     {
7         Co_Task get_return_object()          { return {}; }
8         std::suspend_always initial_suspend() { return {}; }
9         std::suspend_always final_suspend() noexcept { return {}; }
10        void return_void()                   {}
11        void unhandled_exception()           {}
12    };
13 };
14
15 Co_Task coro_work()
16 {
17     co_return;
18 }

```

compiles! We just need to fill in details and implement given functions properly.

There are way too many different ways to implement coroutine task/promise types. There are no constraints and, in general, it all depends on your design and needs. We'll go with an owning coroutine task type:

1. **Co_Task** will own coroutine handle (as in free coroutine in the destructor).
2. Because of the above, **final_suspend()** must suspend always.
3. **Co_Task** will be a “lazy” coroutine, meaning, it's going to be suspended after initial call of **coro_work()/coroutine** function.
4. Because of the above, **initial_suspend()** must suspend.
5. Because coroutine is suspended initially, **Co_Task** needs to expose **resume()** or similar function to run a coroutine.

For now, let's proceed with implementation. Since we own coroutine, our **Co_Task**

needs to have destructor, should be move-only:

```
1 struct Co_Task
2 {
3     struct promise_type;
4     using co_handle = std::coroutine_handle<promise_type>;
5
6     struct promise_type
7     {
8         Co_Task get_return_object()
9         {
10             return Co_Task{co_handle::from_promise(*this)};
11         }
12         // ...
13     };
14
15     Co_Task(co_handle coro)
16         : _coro{coro} {}
17     Co_Task(Co_Task&& rhs) noexcept
18         : _coro{std::exchange(rhs._coro, {})} {}
19     Co_Task(const Co_Task&) = delete;
20     ~Co_Task() noexcept
21     {
22         if (_coro)
23         {
24             _coro.destroy();
25         }
26     }
27
28     co_handle _coro;
29 };
```

In short, when we call `coro_work()`, compiler creates `Co_Task::promise_type` and invokes `get_return_object()` to be able to return an instance of `Co_Task` to the user. Here, in `get_return_object()` there is a way to get an access to `std::coroutine_handle<>` - the only way to interact with just allocated coroutine. Once `Co_Task` is created, we return it to the user. It's **up to the user** to manage `std::coroutine_handle<>`. In our case, we own just created coroutine, hence if `Co_Task` is destroyed, we assume coroutine is in suspended state and destroy it too.

Writing down the rest of functions:

```
1 std::suspend_always promise_type::initial_suspend()
2 {
3     return {};
4 }
```

```

5
6  std::suspend_always promise_type::final_suspend() noexcept
7  {
8      return {};
9  }
10
11 void promise_type::return_void()
12 {
13     // yeah, we return void. Nothing to do
14 }
15
16 void promise_type::unhandled_exception()
17 {
18     // crash, no exceptions handling
19     assert(false);
20 }

```

we can test the basics:

```

1 Co_Task coro_work()
2 {
3     std::println("inside coro_work");
4     co_return;
5 }
6
7 int main()
8 {
9     Co_Task coro = coro_work();
10 }

```

which runs and... prints nothing since our coroutine is created and immediately suspended even before executing first print.

Lets expose `resume()` for our `Co_Task` and use it:

```

1 void Co_Task::resume()
2 {
3     assert(!_coro);
4     assert(!_coro.done());
5     _coro.resume();
6 }
7
8 Co_Task coro_work()
9 {
10     std::println("inside coro_work");
11     co_return;
12 }

```

```

13
14 int main()
15 {
16     std::println("-- before coro_work()");
17     Co_Task coro = coro_work();
18     std::println("-- after coro_work()");
19     coro.resume();
20     std::println("-- after resume()");
21 }

```

which prints:

```

-- before coro_work()
-- after coro_work()
inside coro_work
-- after resume()

```

5.2 #C++ coroutines, basic await

[source code](#).

Given that we can have simplest coroutine, what does it take to `co_await`? Lets try to compile:

```

1 struct Co_CurlAsync {};
2
3 Co_Task coro_work()
4 {
5     co_await Co_CurlAsync{};
6     co_return;
7 }

```

MSVC complains:

```

1 main.cc(73,26): error C2039: 'await_ready': is not a member of 'Co_CurlAsync'
2 main.cc(73,26): error C2039: 'await_suspend': is not a member of 'Co_CurlAsync'
3 main.cc(70,26): error C2039: 'await_resume': is not a member of 'Co_CurlAsync'

```

So `co_await` requires “awaiter” to have those 3 functions. We can think about awaiter as something that:

1. knows if some operation is ready or not (`..._ready`)
2. knows how to resume coroutine later (`..._suspend`)
3. knows how to get the result of awaited operation (`..._resume`)

The compiler asks awaiter, specifically, `Co_CurlAsync` with `bool await_ready()` if operation is done/ready or is in progress. If awaiter returns false, the compiler switches current coroutine state to “suspended” and invokes awaiter’s `await_suspend(std::coroutine_handle<> coro)` customization point which

allows to remember currently suspended coroutine `coro` handle, to call `.resume()` later, once operation is done. Once coroutine is resumed, compiler asks for a value from last awaiter responsible for suspend.

In short, we can have `Co_CurlAsync` awaiter that tells that (1) operation is not ready yet (2) on suspend, resumes coroutine immediately and (3) returns nothing:

```
1 struct Co_CurlAsync
2 {
3     bool await_ready()
4     {
5         return false;
6     }
7
8     void await_suspend(std::coroutine_handle<> coro)
9     {
10        std::println("-- inside suspend, resuming immediately");
11        coro.resume();
12    }
13
14    void await_resume()
15    {
16        std::println("-- resume");
17    }
18 };
19
20 Co_Task coro_work()
21 {
22     std::println("before co_await");
23     co_await Co_CurlAsync{};
24     std::println("after co_await");
25     co_return;
26 }
27
28 int main()
29 {
30     Co_Task coro = coro_work();
31     coro.resume();
32 }
```

which prints:

```
before co_await
-- inside suspend, resuming immediately
-- resume
after co_await
```

Now, on suspend, we did nothing, but immediately resumed coroutine. But we also could start an async operation and, on finish, resume the coroutine.

5.3 #C++ coroutines, await callback with a crash

[source code](#).

Lets continue implementing Co_CurlAsync above, in short:

```
1 struct Co_CurlAsync
2 {
3     CURL_Async _curl_async{};
4     std::string _url;
5     std::coroutine_handle<> _coro;
6     std::string _response;
7
8     bool await_ready()
9     { // 1. CURL_async_get() is not yet started, force coroutine suspend:
10         return false;
11     }
12
13     void await_suspend(std::coroutine_handle<> coro)
14     { // 2. remember coroutine handle, start request, resume on finish:
15         _coro = coro;
16
17         CURL_async_get(_curl_async, _url, this
18             , [](void* user_data, std::string response)
19             {
20                 Co_CurlAsync& self = *static_cast<Co_CurlAsync*>(user_data);
21                 self._response = std::move(response);
22                 self._coro.resume();
23             });
24     }
25
26     std::string await_resume()
27     { // 3. after resume, return response:
28         return std::move(_response);
29     }
30 };
31
32 Co_CurlAsync CURL_await_get(CURL_Async curl_async, const std::string& url)
33 {
34     return Co_CurlAsync{._curl_async = curl_async, ._url = url};
35 }
```

So, now `co_await CURL_await_get(..., "url")` should compile and kind-a work. As always, there are few moving part.

When coroutine function (represented as `std::coroutine_handle<>`) `co_await`s our CURL awaiter - `Co_CurlAsync`, we:

1. force whole coroutine to suspend, since we return false from `await_ready()`
2. this is needed so compiler invokes `await_suspend()` and gives us a handle to currently awaiting coroutine, so we can (a) start request and (b) resume coroutine with a call to `coro.resume()`
3. finally, once request is complete, we can return the `_response` from `await_resume()`

There is one big issue there: what if we start a request with `CURL_async_get()`, coroutine suspends, BUT user discards `Co_Task` value that destroys coroutine, making `std::coroutine_handle<>` we remembered - dangling? There are several possible solutions, but lets see the current code in action by writing our `main()` function:

```
1 Co_Task coro_main(CURL_Async curl_async)
2 {
3     const std::string response = co_await CURL_await_get(
4         curl_async, "localhost:5001/file1.txt");
5
6     std::println("{} ", response);
7     co_return;
8 }
9
10 int main()
11 {
12     CURL_Async curl_async = CURL_async_create();
13     Co_Task task = coro_main(curl_async);
14     task.resume();
15     while (task.is_in_progress())
16     {
17         CURL_async_tick(curl_async);
18     }
19     CURL_async_destroy(curl_async);
20 }
```

Here, we setup `CURL_Async`, as usual, and drive the loop until coroutine is in progress:

```
1 bool Co_Task::is_in_progress() const
2 {
3     assert(!_coro);
4     return !_coro.done();
5 }
```

Running the sample should print:

Fox

However, that works because we wait for coroutine until full complete. If we discard `Co_Task` too early, there is going to be a crash:

```
1  int main()
2  {
3      CURL_Async curl_async = CURL_async_create();
4
5      {
6          Co_Task task = coro_main(curl_async);
7          task.resume(); // run
8      } // **destroy**
9
10     while (true)
11     {
12         CURL_async_tick(curl_async); // resume coroutine from there
13     }
14     CURL_async_destroy(curl_async);
15 }
```

It happens because `CURL_async_get()` callback remembers 2 pointers:

1. `this` pointer to `Co_CurlAsync/awaiter` which is owned by coroutine frame
2. and `coroutine_handle<>` itself, which we destroy BEFORE `CURL_async_get()` finish.

In short, we start request, then `.destroy()` coroutine, then try to resume dangling coroutine inside a callback with a call to `.resume()` even using stale pointer to awaiter (user data in the callback).

There are several solutions, few of them:

1. Don't own and don't destroy coroutine inside `Co_Task` destructor (.. in a multiple ways).
2. Delay coroutine destroy if there are live references to it.
3. Be able to cancel `CURL_async_get()` request if coroutine/awaiter is destroyed.
4. Ensure that callback has a safe way to detect dead coroutine and do nothing.

1st solution could be the best but changes completely the semantics of `Co_Task`, does not allow to easily have `Co_Task<T>` that return some value and requires to be able to change `Co_Task` internals.

2nd solution is similar in the sense that it also requires `Co_Task` changes and the code around.

3rd solution requires changes to our basic C-style callback API which we assume we can't do (since, otherwise, the interface is more advanced).

4th solution is the most inefficient and requires no changes neither in `Co_Task`

nor in callback API.

5.4 #C++ coroutines, await callback

[source code](#).

Lets fix the problematic part in a simple way:

```
1 // struct Co_CurlAsync ...
2 void await_suspend(std::coroutine_handle<> coro)
3 { // 2. remember coroutine handle, start request, resume on finish:
4     _coro = coro;
5
6     CURL_async_get(_curl_async, _url
7         , this // ** HERE
8         , [](void* user_data, std::string response)
9     {
10         Co_CurlAsync& self = *static_cast<Co_CurlAsync*>(user_data);
11         self._response = std::move(response);
12         self._coro.resume();
13     });
14 }
```

For a “happy” path, when `CURL_async_get()` completes before coroutine destruction, the flow is:

- 1) create `Co_CurlAsync`, invoke `CURL_async_get`
- 2) invoke callback (access this/coroutine)
- 3) destroy `Co_CurlAsync`

For a “bad” path, when coroutine is destroyed while we have `CURL_async_get` in progress, the flow is:

- 1) create `Co_CurlAsync`, invoke `CURL_async_get`
- 2) destroy `Co_CurlAsync`
- 3) invoke callback (access this/dead coroutine)

Lets allocate a separate object that can outlive the coroutine/await. There is an assumption that `CURL_async_get()` callback is always going to be invoked. Given this, we:

- A) allocate `WaitState` object - right before `CURL_async_get()`
- B) pass it to the callback instead of `this`
- C) destroy allocated object inside callback

This way we guarantee that the object is always alive while request is in progress and its lifetime is bound the the request itself and nothing else:

```
1 _wait_state = new(std::nothrow) WaitState{._self = this};
2 assert(_wait_state);
```

```

3
4  CURL_async_get(_curl_async, _url
5      , _wait_state
6      , [](void* user_data, std::string response)
7  {
8      WaitState* wait_state = static_cast<WaitState*>(user_data);
9      assert(wait_state);
10     if (wait_state->_self)
11     {
12         Co_CurlAsync& self = *wait_state->_self;
13         self._wait_state = nullptr;
14         self._response = std::move(response);
15         self._coro.resume();
16     }
17     // else: Co_CurlAsync/coroutine is dead
18     delete wait_state;
19 });

```

WaitState is just a struct that has a reference to Co_CurlAsync:

```

1  struct Co_CurlAsync
2  {
3      struct WaitState
4      {
5          Co_CurlAsync* _self = nullptr;
6      };
7      WaitState* _wait_state = nullptr;

```

When coroutine/Co_CurlAsync is destroyed, we need to mark WaitState reference to it as null so CURL_async_get() knows it's not alive:

```

1  ~Co_CurlAsync()
2  {
3      if (_wait_state)
4      { // CURL_async_get() is still in progress
5          assert(_wait_state->_self == this);
6          _wait_state->_self = nullptr; // dead
7      }
8      // else: CURL_async_get() is already completed
9  }

```

That's how you write inefficient coroutine types for a systems that know nothing about coroutines. When doing simple call:

```

1  const std::string response = co_await CURL_await_get(
2      curl_async, "localhost:5001/file1.txt");

```

we:

1. allocate coroutine frame itself
2. `CURL_await_get` allocates `WaitState`
3. `CURL_async_get` allocates `std::string` to write a response
4. `CURL_async_get` allocates `std::function` for a generic callback
5. `CURL_async_get` allocates `std::unordered_map` node to remember what to call when
6. .. and probably something else (CURL internals, etc)

“simple” `CURL_async_get()` implementation alone brings 3 allocations. “simple” `co_await CURL_async_get()` brings 2 more separate allocations.

With CURL scheduler that **knows** about coroutines and few more optimizations and limitations, this number of allocations can go down to amortized 0:

- CURL scheduler preallocates up to N max requests
- request itself knows how to store the response and coroutine-callback inline
- coroutine itself re-uses memory pool for up to N max active coroutines.

Anyway, we now can co-await requests with a nice syntax:

```

1  static Co_Task coro_main(CURL_Async curl_async)
2  {
3      const std::string r1 = co_await CURL_await_get(
4          curl_async, "localhost:5001/file1.txt");
5      const std::string r2 = co_await CURL_await_get(
6          curl_async, "localhost:5001/file2.txt");
7      std::println("{} ", r1);
8      std::println("{} ", r2);
9      co_return;
10 }
11
12 int main()
13 {
14     CURL_Async curl_async = CURL_async_create();
15     Co_Task task = coro_main(curl_async);
16     task.resume();
17     while (task.is_in_progress())
18     {
19         CURL_async_tick(curl_async);
20     }
21     CURL_async_destroy(curl_async);
22 }
```

Note, however, we run 2 requests one after another: second request starts when first `co_await` ends. There is no concurrency..

5.5 #coroutine task that holds a return value

[source code](#).

To be more useful, we'd like to be able return something out of coroutine:

```
1 static Co_Task<int> coro_main()
2 {
3     co_return 3;
4 }
5
6 int main()
7 {
8     Co_Task<int> task = coro_main();
9     task.resume();
10    std::println("{} ", task.get_once());
11 }
```

It requires implementing promise_type return_value() and return_void() customization points. The only complication is that we need to handle Co_Task<void>. To do so, lets start with promise_return<T>:

```
1 template<typename T>
2 struct promise_return
3 {
4     std::variant<std::monostate, T> _value;
5     template<typename U>
6     void return_value(U&& v) noexcept
7     {
8         _value.template emplace<1>(std::forward<U>(v));
9     }
10    const T& get() const noexcept
11    {
12        assert(_value.index() == 1);
13        return std::get<1>(_value);
14    }
15    T&& get_once() noexcept
16    {
17        const T& v = static_cast<const promise_return&>(*this).get();
18        return std::move(const_cast<T&>(v));
19    }
20 };
21
22 template<>
23 struct promise_return<void>
24 {
25     void return_void() noexcept
26     {
27     }
28     void get() const noexcept
29     {
```

```

30     }
31     void get_once() noexcept
32     {
33     }
34 };
35
36 template<typename T>
37 struct promise_return<T&>
38 {
39     static_assert(sizeof(T) == 0, "we do not support Co_Task<T&>");
40 };

```

promise_return for void injects return_void(), get() returns nothing. promise_return for any type T injects return_value() and remembers the value. std::variant is used to handle non-default-constructible types and, in general, construct a value only at the point of the actual return/co_return.

With the help of promise_return, promise_type remains the same as our initial version:

```

1  template<typename T>
2  struct Co_Task
3  {
4      struct promise_type;
5      using co_handle = std::coroutine_handle<promise_type>;
6
7      struct promise_type : promise_return<T>
8      {
9          Co_Task get_return_object() noexcept
10         {
11             return Co_Task{co_handle::from_promise(*this)};
12         }
13         std::suspend_always initial_suspend() noexcept
14         {
15             return {};
16         }
17         std::suspend_always final_suspend() noexcept
18         {
19             return {};
20         }
21         void unhandled_exception() noexcept
22         {
23             // crash, no exceptions handling
24             assert(false);
25         }
26     };

```

```

27
28 // ...
29 decltype(auto) get() const
30 {
31     assert(_coro);
32     return _coro.promise().get();
33 }
34 decltype(auto) get_once() const
35 {
36     assert(_coro);
37     return _coro.promise().get_once();
38 }
39
40 co_handle _coro;
41 };

```

Now `Co_Task<T>` works:

```

1 static Co_Task<int> coro_main()
2 {
3     co_return 3;
4 }

```

5.6 #awaiting coroutine task

[source code](#).

Now, given `Co_Task<T>`, we need to await for it somehow:

```

1 static Co_Task<int> coro_get()
2 {
3     co_return 3;
4 }
5
6 static Co_Task<int> coro_main()
7 {
8     const int v = co_await coro_get();
9     co_return v;
10 }

```

There are 2 important implementation bits to know:

- [Symmetric Transfer](#)
- `final_suspend()` can return **custom** awaitable

`final_suspend()` allows to inject custom code for the moment of coroutine END, so we can do something at that point in time:

```

1 struct promise_type : promise_return<T>
2 {
3     auto final_suspend() noexcept
4     {
5         struct Final_Await : std::suspend_always
6         {
7             void await_suspend(co_handle self_coro) noexcept
8             {
9                 // **HERE**
10            }
11        };
12        return Final_Await{};
13    }

```

Given a simple coroutine:

```

1 static Co_Task<int> coro_get()
2 {
3     co_return 3;
4 } // <-- final_suspend()

```

our `Final_Await`, `await_suspend()` gets invoked after `co_return`, passing a coroutine that is about to end. This is exactly what we need to notify some other waiting code that we are done. We could invoke a callback... BUT with a symmetric transfer, `await_suspend()` can return a **next** coroutine that must be executed:

```

1 struct promise_type : promise_return<T>
2 {
3     std::coroutine_handle<> _waiting_coro = std::noop_coroutine();
4
5     auto final_suspend() noexcept
6     {
7         struct Final_Await : std::suspend_always
8         {
9             std::coroutine_handle<> await_suspend(co_handle self_coro) noexcept
10            {
11                return self_coro.promise()._waiting_coro;
12            }
13        };
14        return Final_Await{};
15    }

```

Here, if no one waits for us, we return `std::noop_coroutine()` that does nothing. Otherwise, if `_waiting_coro` was set, we end the execution of our coroutine and resume any other coroutine that was waiting for us. To setup `_waiting_coro`, we implement awaitable interface for `Co_Task` itself:

```

1  template<typename T>
2  struct Co_Task
3  {
4      bool await_ready()
5      {
6          assert(is_in_progress());
7          return false;
8      }
9      // intentionally auto, not decltype(auto)
10     auto await_resume()
11     {
12         return get_once();
13     }
14     std::coroutine_handle<> await_suspend(std::coroutine_handle<> waiting_coro)
15     {
16         _coro.promise()._waiting_coro = waiting_coro;
17         return _coro; // resume us
18     }
19
20     co_handle _coro;
21 };

```

It's important to understand what is `_coro` and what is `waiting_coro`. We have 3 moving parts (1) `Final_Await::await_suspend()` with `self_coro`, (2) `Co_Task::await_suspend()` `waiting_coro` and (3) `Co_Task _coro` member itself:

```

1  struct Co_Task
2  {
3      [1] ... promise_type::Final_Await::await_suspend(co_handle self_coro) noexcept
4      {
5          return self_coro.promise()._waiting_coro;
6      }
7      [2] ... await_suspend(std::coroutine_handle<> waiting_coro)
8      {
9          _coro.promise()._waiting_coro = waiting_coro;
10         return _coro; // resume us
11     }
12     [3] co_handle _coro;
13 };

```

Given our awaiting code sample:

```

1  static Co_Task<int> coro_get()
2  {
3      co_return 3;
4  }
5

```

```

6 static Co_Task<int> coro_main()
7 {
8     co_return co_await coro_get();
9 }

```

We have 2 coroutines created, where:

1. `_coro` from part [3] is `coro_get()`
2. `waiting_coro` from part [2] `await_suspend()` is our `coro_main()`; and
3. `self_coro` from part [1] is `coro_get()` again that gets finished.

So, we

1. create `coro_main()`
2. create `coro_get()`
3. then `await_suspend()` on `coro_get()` from within `coro_main()`
4. that remembers that `_waiting_coro` is `coro_main()`
5. resumes `coro_get()` execution (by returning it, symmetric transfer); and then
6. we finish `coro_get()` and its `final_suspend()` resumes `_waiting_coro` which is `coro_main()`.

That way `coro_get()` was scheduled to be executed because we `co_await` it from within `coro_main()` and `coro_main()` was executed 2nd time because `coro_get()` was completed.

One note on the `co_await` semantic and ownership: see how for `await_resume()` we use `get_once()` to move the value out awaited task. In general, when `co_awaiting`, we consume the task and it can't be used after `co_await`. So the next code should be disallowed:

```

1 static Co_Task<int> coro_main()
2 {
3     Co_Task<int> t = coro_get(1);
4     co_await t; // t is lvalue
5     t.get(); // error
6 }

```

To enforce that, we use operator `co_await()` `&&` with `&&` ref-qualifier; see also [C++ Coroutines: Understanding operator co_await](#):

```

1 struct Co_Await
2 {
3     Co_Task<T> _task;
4
5     bool await_ready()
6     {
7         assert(_task.is_in_progress());
8         return false;

```

```

9     }
10    // intentionally auto, not decltype(auto)
11    auto await_resume()
12    {
13        return _task.get_once();
14    }
15    std::coroutine_handle<> await_suspend(std::coroutine_handle<> waiting_coro)
16    {
17        _task._coro.promise()._waiting_coro = waiting_coro;
18        return _task._coro; // resume us
19    }
20 };
21
22 Co_Await operator co_await() &&
23 {
24     // consume this.
25     return Co_Await{._task{std::move(*this)}};
26 }

```

Now we can co_await our tasks multiple times:

```

1 static Co_Task<int> coro_get(int v)
2 {
3     co_return v;
4 }
5
6 static Co_Task<int> coro_main()
7 {
8     const int v1 = co_await coro_get(2);
9     const int v2 = co_await coro_get(3);
10    co_return (v1 + v2);
11 }
12
13 int main()
14 {
15     Co_Task<int> task = coro_main();
16     task.resume();
17     std::println("{} ", task.get_once());
18 }

```

Note, how we still execute coro_get(2) first then coro_get(3) second. There is still no way to launch those 2 coroutines concurrently and wait for both of them:

```

1 static Co_Task<int> coro_main()
2 {
3     auto [v1, v2] = co_await CO_await_all(coro_get(2), coro_get(3));

```

```

4     co_return (v1 + v2);
5 }

```

5.7 #waiting for multiple coroutines

[source code](#).

Lets implement awaiting for multiple Co_Tasks:

```

1 static Co_Task<int> coro_main()
2 {
3     auto [v1, v2] = co_await CO_await_all(coro_get(2), coro_get(3));
4     co_return (v1 + v2);
5 }

```

Note how we return a tuple of all awaited results (since our coroutine task can never fail). Also, note how this is different to sequential `co_await` since all children coroutine tasks are started at the point of `await` and executed concurrently:

```

1 static Co_Task<void> coro_main()
2 {
3     int v1 = co_await coro_get(2);
4     int v2 = co_await coro_get(3); // runs strictly AFTER first task
5 }

```

To await N tasks, our parent coroutine/task needs to be resumed only when last of that N tasks completes. To achieve this, we introduce a counter that signals how many tasks are still in progress. So our `Final_Await` can resume the parent only when the wait count is zero. See the old implementation:

```

1 struct promise_type : promise_return<T>
2 {
3     std::coroutine_handle<> _waiting_coro = std::noop_coroutine();
4
5     auto final_suspend() noexcept
6     {
7         struct Final_Await : std::suspend_always
8         {
9             std::coroutine_handle<> await_suspend(co_handle self_coro) noexcept
10             {
11                 return self_coro.promise()._waiting_coro;
12             }
13         };
14         return Final_Await{};
15     }

```

and compare to a new one:

```

1 struct promise_type : promise_return<T>
2 {
3     std::int32_t* _wait_count = nullptr;
4     co_handle _waiting_coro;
5
6     std::coroutine_handle<> Final_Await::await_suspend(co_handle self_coro) noexcept
7     {
8         promise_type& self = self_coro.promise();
9         if (self._waiting_coro)
10         {
11             assert(self._wait_count);
12             std::int32_t& wait_count = *self._wait_count;
13             wait_count -= 1;
14             assert(wait_count >= 0);
15             if (wait_count == 0)
16             {
17                 return self._waiting_coro;
18             }
19             // else: we are not the last task, do nothing.
20         }
21         // else: no one was awaiting us.
22         return std::noop_coroutine();
23     }

```

When Co_Task ends, we:

1. check if someone waits for us
2. decrement wait counter by 1
3. see if we are the last one and resume awaiting coroutine

With this change, awaiting a single Co_Task requires setting the counter:

```

1 struct Co_Await
2 {
3     Co_Task<T> _task;
4     std::int32_t _wait_count = 1;
5     std::coroutine_handle<> Co_Await::await_suspend(
6         std::coroutine_handle<> waiting_coro) noexcept
7     {
8         promise_type& promise = _task._coro.promise();
9         promise._wait_count = &_amp;wait_count; // 1
10        promise._waiting_coro = waiting_coro;
11        return _task._coro; // resume us
12    }
13 };

```

For awaiting of N coroutines/tasks, we start with CO_await_all():

```

1  template<typename... Ts>
2  static auto CO_await_all(Co_Task<Ts>&&... tasks)
3  {
4      static_assert(sizeof...(Ts) > 0);
5      using Is = std::index_sequence_for<Ts...>;
6      return Co_Await_All<Is, Ts...>{._tasks{std::move(tasks)...}};
7  }

```

It accepts variadic number of any tasks, consumes all of them and returns an awaiter - Co_Await_All. We store all of the Co_Tasks and when wait completes, return a tuple of all of the results:

```

1  template<auto... Is, typename... Ts>
2  struct Co_Await_All<std::index_sequence<Is...>, Ts...>
3  {
4      std::tuple<Co_Task<Ts>...> _tasks;
5      std::int32_t _wait_count = sizeof...(Ts);
6      bool await_ready()
7      {
8          // assert(_tasks[I].is_in_progress());
9          return false;
10     }
11     auto await_resume()
12     {
13         return std::tuple<Ts...>{std::get<Is>(_tasks).get_once()...};
14     }
15 };

```

When awaiting starts, we (a) setup wait_count=N and (b) resume all of the tasks, also linking awaiting coroutine - as in the regular single-task await:

```

1  void Co_Await_All::await_suspend(std::coroutine_handle<> waiting_coro)
2  {
3      auto handle = [&](auto& Task)
4      {
5          auto& promise = Task._coro.promise();
6          promise._wait_count = &_amp;wait_count;
7          promise._waiting_coro = waiting_coro;
8          Task._coro.resume();
9      };
10
11     (handle(std::get<Is>(_tasks)), ...);
12 }

```

Finally, we can await several concurrently running Co_Tasks:

```

1  static Co_Task<int> coro_get(int v)
2  {

```

```

3     co_return v;
4 }
5
6 static Co_Task<int> coro_main()
7 {
8     auto [v1, v2] = co_await CO_await_all(coro_get(2), coro_get(3));
9     co_return (v1 + v2);
10 }
11
12 int main()
13 {
14     Co_Task<int> task = coro_main();
15     task.resume();
16     std::println("{} ", task.get_once());
17 }

```

5.8 #waiting for multiple CURL requests with tasks

[source code.](#)

We already built `CURL_await_get()` that awaits CURL request within a coroutine. With `CO_await_all()` API, we can await for multiple concurrent CURL requests by wrapping every request into a separate coroutine task:

```

1 Co_Task<std::string> CURL_coro_get(CURL_Async curl_async, std::string url)
2 {
3     co_return co_await CURL_await_get(curl_async, url);
4 }
5
6 Co_Task<void> coro_main(CURL_Async curl_async)
7 {
8     auto [r1, r2] = co_await CO_await_all(
9         CURL_coro_get(curl_async, "localhost:5001/file1.txt"),
10        CURL_coro_get(curl_async, "localhost:5001/file2.txt"),
11    );
12    std::println("{} ", r1);
13    std::println("{} ", r2);
14 }

```

That's.. all. Note, however, 3 separate coroutines are allocated (one for `coro_main` and two for `CURL_coro_get`). We can do less generic coroutine awaiter that skips intermediate coroutines.

5.9 #waiting for multiple CURL request with custom awaitable

[source code.](#)

Instead of `CURL_coro_get()` + `CO_await_all()`:

```
1 Co_Task<std::string> CURL_coro_get(CURL_Async curl_async, std::string url)
2 {
3     co_return co_await CURL_await_get(curl_async, url);
4 }
5 // ...
6 auto [r1, r2] = co_await CO_await_all(
7     CURL_coro_get(curl_async, "localhost:5001/file1.txt"),
8     CURL_coro_get(curl_async, "localhost:5001/file2.txt"),
9 );
```

we can make a separate, specialized awaiter, like `CURL_await_get_many()`:

```
1 auto [r1, r2] = co_await CURL_await_get_many(curl_async,
2     "localhost:5001/file1.txt",
3     "localhost:5001/file2.txt"
4 );
```

that skips intermediate `CURL_coro_get()` coroutine.

`CURL_await_get_many()` is almost the same as `CURL_await_get()` implementation:

```
1 template<typename... URLs>
2     requires std::conjunction_v<
3         std::is_constructible<std::string, URLs&&>...>
4 auto CURL_await_get_many(CURL_Async curl_async, URLs... urls)
5 {
6     Co_CurlAsyncMany<std::index_sequence_for<URLs...>> awaiter;
7     awaiter._curl_async = curl_async;
8     awaiter.setup_urls(std::move(urls)...);
9     return awaiter;
10 }
```

Ignoring templates (to be able to accept variadic number of URLs), we create `Co_CurlAsyncMany` awaitable that knows how to wait for N requests:

```
1 template<typename Is>
2 struct Co_CurlAsyncMany;
3
4 template<auto... Is>
5 struct Co_CurlAsyncMany<std::index_sequence<Is...>>
6 {
7     struct WaitState
8     {
9         std::int32_t _active_count = 0;
10         Co_CurlAsyncMany* _self = nullptr;
11     };
```

```

12
13     static constexpr std::size_t Count = sizeof...(Is);
14     CURL_Async _curl_async{};
15     std::coroutine_handle<> _coro;
16
17     WaitState* _wait_state = nullptr;
18     std::array<std::string, Count> _urls;
19     std::array<std::string, Count> _responses;
20
21     template<typename... URLs>
22     void setup_urls(URLs... urls)
23     {
24         ((_urls[Is] = std::move(urls)), ...);
25     }
26
27     // ...
28 };

```

Here, we remember all of URLs and have an `std::array<std::string, Count>` for all of responses. Our `WaitState` passed to `CURL_async_get()` callback also has a counter for active requests since we want to complete only when last request is finished:

```

1 bool Co_CurlAsyncMany::await_ready() noexcept
2 {
3     return false;
4 }
5
6 void Co_CurlAsyncMany::await_suspend(std::coroutine_handle<> coro) noexcept
7 {
8     _coro = coro;
9     _wait_state = new(std::nothrow) WaitState;
10    assert(_wait_state);
11    _wait_state->_active_count = Count;
12    _wait_state->_self = this;
13
14    auto handle = [&<auto I>(std::integral_constant<std::size_t, I>)
15    {
16        CURL_async_get(_curl_async, _urls[I]
17            , _wait_state
18            , [](void* user_data, std::string response)
19            {
20                WaitState* wait_state = static_cast<WaitState*>(user_data);
21                assert(wait_state);
22                if (on_response<I>(*wait_state, std::move(response)))
23                {

```

```

24         delete wait_state;
25     }
26     });
27 };
28
29     (handle(std::integral_constant<std::size_t, Is>{}), ...);
30 }
31
32 Co_CurlAsyncMany::~Co_CurlAsyncMany() noexcept
33 {
34     if (_wait_state)
35     {
36         assert(_wait_state->_self == this);
37         _wait_state->_self = nullptr; // dead
38     }
39 }
40
41 std::array<std::string, Count> Co_CurlAsyncMany::await_resume() noexcept
42 {
43     return std::move(_responses);
44 }

```

Basically, we:

1. start N requests; and
2. clean-up `wait_state` only for a last response
3. `wait_state` logic/lifetime handling is the same as is for `Co_CurlAsync`

Finally, `on_response<I>()` is also largely the same as `Co_CurlAsync`: (a) we decrement active requests count, (b) see if coroutine is still alive, (c) write a response to proper place and (c) resume awaiting coroutine if we are the last response:

```

1  template<auto I>
2  static bool Co_CurlAsyncMany::on_response(
3      WaitState& wait_state, std::string&& response)
4  {
5      wait_state._active_count -= 1;
6      assert(wait_state._active_count >= 0);
7
8      if (wait_state._self)
9      {
10         Co_CurlAsyncMany& self = *wait_state._self;
11         self._responses[I] = std::move(response);
12         if (wait_state._active_count == 0)
13         {
14             self._wait_state = nullptr;

```

```

15         self._coro.resume();
16     }
17 }
18 // else: Co_CurlAsyncMany/coroutine is dead
19
20 if (wait_state._active_count == 0)
21 {
22     return true; // done
23 }
24 return false;
25 }

```

With that in mind, we can do:

```

1 Co_Task<void> coro_main(CURL_Async curl_async)
2 {
3     auto [r1, r2] = co_await CURL_await_get_many(curl_async,
4         "localhost:5001/file1.txt",
5         "localhost:5001/file2.txt"
6     );
7     std::println("{} ", r1);
8     std::println("{} ", r2);
9 }

```

which is slightly different to a more generic version:

```

1 Co_Task<void> coro_main(CURL_Async curl_async)
2 {
3     auto [r1, r2] = co_await CO_await_all(
4         CURL_coro_get(curl_async, "localhost:5001/file1.txt"),
5         CURL_coro_get(curl_async, "localhost:5001/file2.txt"),
6     );
7     std::println("{} ", r1);
8     std::println("{} ", r2);
9 }

```

For the rest of the sample code, we'll go using `CO_await_all()` version.

[#coroutines on top polling tasks](#)

[6 #building Fibers API](#)

[source code](#), [sample app](#).

(Win32) Fibers materials:

- [Using Fibers](#)

- [Fibers: the Most Elegant Windows API](#)

In short, we'd like to be able to write something like this:

```
1  const std::string response = FF_await_get(
2      curl_async, "localhost:5001/file1.txt");
3  // use `response` as a usual variable, no callbacks
```

to make an asynchronous request. No callbacks, no special keywords.

While “Using Fibers” example above is nice, it’s still overly complicated to get basic idea in a simpler form.

Going with Win32 Fibers, short intro is:

- fibers allow to suspend and resume execution at any given point inside a function
- they are stackful coroutines, as opposed to C++20 coroutines that are stackless
- they implement symmetric coroutines (same as C++20 coroutines); we’ll build asymmetric coroutines on top of Fibers
- it should be trivial to ifdef POSIX implementation using deprecated `ucontext.h`

and the general idea is:

- you create a fiber with `::CreateFiber()` API; it’s suspended
- you switch to/activate/run a fiber with `::SwitchToFiber()` call
- you can switch only between fibers; so everything must be a fiber

Last point is more specific to Win32 API:

- from within a `main()` entry point we are in a thread context
- once `::CreateFiber()` gets you a fiber to switch to, (main) thread needs to be converted to a fiber with a call to `::ConvertThreadToFiber()`

But, ignoring thread-to-fiber conversion, we just (a) create N fibers and (b) switch an execution between them. Worth mentioning: fibers still execute withing a thread context, meaning - context switches (between threads) still happen while fiber is executing.

Below, we go with a `Fiber` class that encapsulates all the system APIs above. We’d like to have a working code that may look like this:

```
1  void Fiber::run()
2  {
3      std::println("fiber1");
4      suspend();
5      std::println("fiber2");
6  }
7
8  int main()
```

```

9  {
10     Fiber fiber;
11     std::println("main1");
12     fiber.resume();
13     std::println("main2");
14     fiber.resume();
15     std::println("main3");
16 }

```

and prints:

```

1  main1
2  fiber1
3  main2
4  fiber2
5  main3

```

6.1 #basic Fiber

[source code](#).

We start with a Fiber class that allocates a fiber:

```

1  struct Fiber
2  {
3      void* _fiber = nullptr;
4      void* _parent_fiber = nullptr;
5
6      Fiber(const Fiber&) = delete;
7      Fiber()
8      {
9          _fiber = ::CreateFiber(
10             0 // default stack size
11             , &FiberProc
12             , this); // parameter to FiberProc
13         assert(_fiber);
14     }
15     ~Fiber()
16     {
17         ::DeleteFiber(_fiber);
18         _fiber = nullptr;
19     }

```

To make our lives easier, there are few assumptions and simplifications:

- we are on x64 system, so there is no need to use extended Fibers API
- we assume LPVOID is void* so no Win32 API types are used (great for headers)

- and, given x64, WINAPI/__stdcall can be omitted, so callbacks passed to Win32 API can have simple C++ declarations

That gives us next include:

```

1 #include <Windows.h>
2 // Note: the floating-point state on x86 systems is not preserved.
3 // If there is need to support x86, Fiber's Ex-tended API must be used.
4 #if !defined(_WIN64)
5 # error Fiber implementation does not support x86 systems.
6 #endif
7 #include <type_traits>
8 static_assert(std::is_same_v<LPVOID, void*>);

```

Next, while Win32 Fibers can switch from any one Fiber to any other Fiber, we simplify and implement a Fiber that can be resumed, but when suspends - goes to the point of last resume (its parent). Hence, `void* _parent_fiber`.

`FiberProc()` we pass to `::CreateFiber()` gets a reference to a given `Fiber` instance and runs it. `FiberProc` must never end, hence a while loop and a `suspend()` if a call to `run()` ends:

```

1 static void FiberProc(void* parameter)
2 {
3     assert(parameter);
4     Fiber& self = *static_cast<Fiber*>(parameter);
5     while (true)
6     {
7         self.run();
8         self.suspend();
9     }
10 }

```

To suspend a Fiber is to switch to our parent fiber who did resume us:

```

1 void suspend()
2 {
3     assert(_parent_fiber);
4     void* switch_to_fiber = _parent_fiber;
5     _parent_fiber = nullptr;
6     ::SwitchToFiber(switch_to_fiber);
7 }

```

To resume a Fiber, we simply call `::SwitchToFiber()` for a `_fiber` we created. However, since when suspending, we need to know how to switch back, we remember current fiber as our parent:

```

1 void resume()
2 {

```

```

3     assert(_parent_fiber == nullptr);
4     _parent_fiber = ::GetCurrentFiber();
5     ::SwitchToFiber(_fiber);
6 }

```

Remember, `.resume()` is, basically, a first call to a Fiber from within a main function/thread. That means that `::GetCurrentFiber()` is invoked in a context of `main()`:

```

1 int main()
2 {
3     Fiber fiber;
4     fiber.resume(); // call to ::GetCurrentFiber()??

```

To make that work, specifically for Win32 API, main thread needs to become a Fiber. This is what we do by having a simple RAII class:

```

1 struct Fiber::Boot
2 {
3     Boot(const Boot&) = delete;
4     Boot()
5     {
6         const void* fiber = ::ConvertThreadToFiber(nullptr);
7         assert(fiber);
8     }
9     ~Boot()
10    {
11        const auto ok = ::ConvertFiberToThread();
12        assert(ok);
13    }
14 };

```

Hence, main and/or any other thread that may use Fibers, needs to scope `Fiber::Boot` variable on top:

```

1 int main()
2 {
3     Fiber::Boot _;
4
5     Fiber fiber;
6     std::println("main1");
7     fiber.resume();
8     std::println("main2");
9     fiber.resume();
10    std::println("main3");
11 }

```

It's possible to avoid that by calling `::ConvertThreadToFiber()` on each and

every call to `Fiber::resume()`; choose what you like more.

Given a `main()` above, we:

- create a fiber, which is suspended initially
- print “main1”
- first call to `.resume()` switches us back to `FiberProc` that invokes `Fiber::run()`:

```
1 void Fiber::run()
2 {
3     std::println("fiber1");
4     suspend();
5     std::println("fiber2");
6 }
```

that:

- prints “fiber1”
- suspends itself, which switches back to `main` (our parent fiber)
- `main` prints “main2”, resumes fiber
- fiber prints “fiber2”, exits `run()`, but immediately suspends itself
- `main` prints “main3”

All at once:

```
main1
fiber1
main2
fiber2
main3
```

6.2 #switching between Fibers

[source code](#).

Section above shows how we can switch from a `main` to a different `Fiber`. `Fiber` on it's own, when suspended, switches back to its invoker/resumer. However, instead of `suspend`, `Fiber` can switch execution to a different `Fiber`. While not quite used anywhere else, lets show the possibility. Our `Fiber::run()` must be able to run different code; lets inject any user-defined callback and run it:

```
1 struct Fiber
2 {
3     // For an example:
4     std::function<void ()> _callback;
5     // ...
6 };
7
8 void Fiber::run()
```

```

9  {
10     assert(_callback);
11     _callback();
12 }

```

Where main can now drive 2 Fibers at once:

```

1  void Fiber::run()
2  {
3      assert(_callback);
4      _callback();
5  }
6
7  int main()
8  {
9      Fiber::Boot _;
10
11     Fiber fiber1;
12     Fiber fiber2;
13     fiber1._callback = [&fiber2, self = &fiber1]()
14     {
15         std::println("fiber1: start");
16         fiber2.resume(); // switch to fiber2
17         std::println("fiber1: END");
18         self->suspend(); // switch to main (=our parent)
19     };
20     fiber2._callback = [self = &fiber2]()
21     {
22         std::println("fiber2: start");
23         self->suspend(); // switch back to fiber1 (=our parent)
24         std::println("fiber2: END");
25         self->suspend(); // switch to main (=our parent)
26     };
27     std::println("main1");
28     fiber1.resume();
29     std::println("main2");
30     fiber2.resume();
31     std::println("main3");
32 }

```

we:

- create 2 fibers; they are suspended
- print “main1”
- resume fiber1, that starts run(), that prints “fiber1: start”
- instead of suspend/switch to main, we then resume/switch to fiber2
- fiber2 resume prints “fiber2: start” initially; then

- we suspend fiber2
- that brings us back to our parent = fiber1
- print “fiber1: END”
- fiber1 suspends itself, that switches back to main
- print “main2”
- main resumes fiber2 which executes last print
- fiber2 prints “fiber2: END”, ends execution by suspending itself
- fiber2 suspend brings back to main
- main prints “main3”

which prints:

```
main1
fiber1: start
fiber2: start
fiber1: END
main2
fiber2: END
main3
```

so, while we used Win32 Fibers to implement asymmetric coroutines:

- we can still freely switch between Fiber(s)
- it does not matter if Fiber was resumed/activated from a thread (main) or the other Fiber
- the act of switching to/resuming is to give a possibility to execute
- this is similar to C++20 coroutines with its `.resume()`
- nothing magically “runs” in the background; there should be a scheduler that resumes or switches between fibers; same is true for C++20 coroutines
- we were able to interleave execution of 3 fibers: main, fiber1, fiber2 - all within one system thread; there are no multiple other threads
- fibers are executed concurrently within main thread

Note, how while executing **fiber2** we (a) were resumed from 2 different contexts and (b) changed the parent in meantime, switching to different contexts:

```
1 fiber2._callback = [self = &fiber2]()
2 {
3     // ... from fiber1
4     self->suspend(); // switch back to fiber1 (=our parent)
5     // ... from main
6     self->suspend(); // switch to main (=our parent)
7 };
```

6.3 #tasks for Fibers

C++20 coroutines allow for a function to return a value:

```

1  co::Tast<int> MyCoroutine()
2  {
3      co_await Request();
4      co_return 1;
5  }
6
7  int main()
8  {
9      co::Tast<int> task = MyCoroutine();
10     // use a task
11 }

```

We need to build something similar on top of `Fiber` class:

```

1  int MyFiber()
2  {
3      this_fiber::suspend();
4      return 1;
5  }
6
7  int main()
8  {
9      FiberTask<int> task = FF_async([] { return MyFiber(); });
10     // use a task
11 }

```

Note how:

- any mention of a `Fiber` goes away
- there is nice interface to launch a new `Fiber`
- we can suspend itself within an execution context; and
- `MyFiber()` simply returns naked `int`; there is no need to mark every coroutine with task-like return type.

In addition, under the hood:

- we use `Fibers` pool to preallocate `N` fibers and reuse them
- there is simple `Fibers` scheduler to drive our `Fibers` execution
- use exceptions to allow cancellable fiber tasks

Last one is presented just to showcase one of the possibilities; not required for `CURL_async_get()` `Fiber` wrapper.

6.4 #cancellable `Fiber` task with a generic callback

[source code](#).

First, we start with replacing hardcoded `Fiber::run()` with a generic version that uses next interface:

```

1 // To separate `FiberTask` implementation from an actual `Fiber`.
2 struct FiberCallbackBase
3 {
4     void run()    { return do_run();    }
5     void resume() { return do_resume(); }
6
7     FiberCallbackBase() noexcept = default;
8     FiberCallbackBase(const FiberCallbackBase& rhs) = delete;
9 protected:
10    ~FiberCallbackBase() noexcept = default;
11    virtual void do_run() = 0;
12    virtual void do_resume() {} // optional
13 };

```

Where we allow a user to `run()` anything and also allow to be notified that Fiber is resumed. This is needed to implement a task cancelation: when canceled, user's defined `resume()` throws an exception to unwind everything in a context of fiber execution. Hence, our `run()` implementation becomes:

```

1 static void FiberProc(void* parameter)
2 {
3     Fiber& self = *static_cast<Fiber*>(parameter);
4     while (true)
5     {
6         self.run();
7         self.suspend();
8     }
9 }
10
11 void Fiber::run()
12 {
13     try
14     {
15         _callback->resume();
16         _callback->run();
17     }
18     catch (...)
19     {
20         _exception = std::current_exception();
21     }
22     _callback = nullptr;
23 }

```

where `_callback` and `_exception` are:

```

1 struct Fiber
2 {
3     void* _fiber = nullptr;
4     void* _parent_fiber = nullptr;
5     FiberCallbackBase* _callback = nullptr;
6     std::exception_ptr _exception;
7     // ...
8 };

```

so, when `run()` enters we (a) notify a user it got resumed (first time) and (b) run everything. If exception is thrown, we just remember it and clean-up our current `_callback` - the execution is done and we go to suspend.

What is `_callback`? This is something user can set on a `Fiber` instance allocated from a `FiberPool` (not shown yet). This is going to be done by a `FiberTask` under the hood. For now, we do everything manually:

```

1 void Fiber::set_callback(FiberCallbackBase& callback)
2 {
3     assert(_callback == nullptr);
4     _callback = &callback;
5     _exception = {};
6 }
7
8 struct MyCallback : FiberCallbackBase
9 {
10     virtual void do_run() override
11     {
12         std::println("fiber1");
13     }
14 };
15
16 int main()
17 {
18     Fiber::Boot _;
19
20     Fiber fiber;
21     MyCallback callback;
22     fiber.set_callback(callback);
23     std::println("main1");
24     fiber.resume();
25     std::println("main2");
26 }

```

For now, we just made everything we had before more complicated. But the difference is that `Fiber::run()` now is generic and can run anything user-defined.

To support exceptions (cancellation) - the rest of `FiberCallbackBase` interface - our old implementation for `suspend()` needs to be tweak:

```
1 void suspend()
2 {
3     assert(_parent_fiber);
4     void* switch_to_fiber = _parent_fiber;
5     _parent_fiber = nullptr;
6     ::SwitchToFiber(switch_to_fiber);
7     assert(_callback);
8     _callback->resume();
9 }
```

So once `::SwitchToFiber()` returns - meaning other Fiber was running and we are resumed, we notify a user on a new `resume()`.

Finally, to check that Fiber is doing something (either running or suspended from within a user code), we expose next function:

```
1 bool is_busy() const
2 {
3     if (_exception)
4     {
5         std::rethrow_exception(_exception);
6     }
7     return !!_callback;
8 }
```

It does cover 2 things: (1) allows to see Fiber has valid user callback and (2) allows to throw any Fiber exception to a user. A bit weird, but does the job.

Overall, the Fiber use for a single Task - becomes:

1. allocate a new Fiber from a pool
2. set a new callback
3. resume/run the fiber to an end (`is_busy() == false`)
4. return Fiber to the pool
5. repeat for a new Task

Note, that from within a Task or `FiberCallbackBase`, there is no access to a Fiber. How can we suspend a Task then? Following `std::this_thread` convention, we have:

```
1 namespace this_fiber
2 {
3     void suspend()
4     {
5         assert(::IsThreadAFiber());
6         void* fiber_data = ::GetFiberData();
7         assert(fiber_data);
8     }
9 }
```

```

8     Fiber& self = *static_cast<Fiber*>(fiber_data);
9     self.suspend();
10 }
11 } // namespace this_fiber

```

That allows to simply do `this_fiber::suspend()` to give up task execution and be re-scheduled later.

Linking all the pieces together, low-level Fiber Task may look like this:

```

1 struct MyFiberTask : FiberCallbackBase
2 {
3     bool _cancel = false;
4
5     virtual void do_run() override
6     {
7         std::println("fiber1");
8         this_fiber::suspend();
9         std::println("fiber2");
10    }
11
12    virtual void do_resume() override
13    {
14        if (_cancel)
15        {
16            _cancel = false;
17            throw std::exception("canceled");
18        }
19    }
20 };
21
22 int main()
23 {
24     Fiber::Boot _;
25
26     Fiber fiber;
27     MyFiberTask task;
28     fiber.set_callback(task);
29     std::println("main1");
30     fiber.resume();
31     std::println("main2");
32     task._cancel = true;
33     fiber.resume();
34     std::println("main3");
35 }

```

This is going to be wrapped into nice `FF_async()` interface later. Interesting

bit here is that we cancel fiber task in the middle of its execution and “fiber2” console line is not printed:

```
main1
fiber1
main2
main3
```

6.5 #basic FiberPool

[source code](#).

The idea is simple: fiber task can be small and short-lived, there is no point allocating (and deallocating) a system Fiber every time task is created. Hence, we preallocate a pool of N (=128) system Fibers that are always alive and reuse them. That means:

- there can be up to N Fiber(s) alive; and
- all system resources are allocated once and never released

FiberPool interface may look like this:

```
1 struct FiberPool
2 {
3     using Handle = std::size_t;
4     explicit FiberPool(std::size_t size) noexcept
5
6     Handle allocate(FiberCallbackBase& callback);
7     void free(Handle handle);
8
9     void suspend(Handle handle);
10    void resume(Handle handle);
11    bool is_busy(Handle handle) const;
12
13 private:
14     std::unique_ptr<Fiber[]> _fibers;
15     std::size_t _size = 0;
16 };
```

where `Fiber` instance is hidden from a user and exposed only with a `Handle`. So we can manage an array of fixed size internally:

```
1 explicit FiberPool::FiberPool(std::size_t size) noexcept
2 {
3     assert(size > 0);
4     _fibers.reset(new(std::nothrow) Fiber[size]);
5     assert(_fibers.get());
6     _size = size;
7 }
```

To create a Fiber is to invoke an `allocate()`. Note, we go as dumb and as simple as possible - doing linear search to find first free Fiber. It all could be made better, having basic free list allocator for indices as one way to go about it:

```

1 Handle FiberPool::allocate(FiberCallbackBase& callback)
2 {
3     auto find_first_free_handle = [this]()
4     {
5         for (std::size_t i = 0; i < _size; ++i)
6         {
7             if (_fibers[i]._callback == nullptr)
8             {
9                 return Handle(i);
10            }
11        }
12        assert(false);
13        return Handle(-1);
14    };
15    const Handle handle = find_first_free_handle();
16    Fiber& fiber = _fibers[handle];
17    fiber.set_callback(callback);
18    return handle;
19 }
```

`FiberPool::free()` is, basically, no-op. We assert Fiber returned is in “complete” state - finished execution:

```

1 void FiberPool::free(Handle handle)
2 {
3     assert(handle < _size);
4     assert(_fibers[handle]._callback == nullptr);
5 }
```

Next, `suspend()`, `resume()` and `is_busy()` are just a wrappers around Fiber API, but for a `Handle`:

```

1 void FiberPool::suspend(Handle handle)
2 {
3     assert(handle < _size);
4     _fibers[handle].suspend();
5 }
```

With all this, our previous example running fiber task becomes:

```

1 int main()
2 {
3     Fiber::Boot _;
4     FiberPool fiber_pool{128};
```

```

5
6     MyFiberTask task;
7     FiberPool::Handle fiber = fiber_pool.allocate(task);
8     fiber_pool.resume(fiber);
9     fiber_pool.free(fiber);
10 }

```

Now, let's get rid of manually created `MyFiberTask` and make `FiberTask<T>` possible.

6.6 #FiberTask

[source code](#).

We'd like to be able to write something like this:

```

1  int MyFiber()
2  {
3      this_fiber::suspend();
4      return 1;
5  }
6
7  int main()
8  {
9      FiberTaskScheduler scheduler;
10     FiberTask<int> task = FF_async(scheduler, [] { return MyFiber(); });
11     // ...
12     // scheduler.schedule();
13 }

```

There are several moving parts:

1. we need type-erased implementation of `FiberTask` that can consume any user-defined lambda/callback, work with any return type `T`; and
2. we need a scheduler that knows how to drive tasks/fibers execution

`FiberTask<T>` allocates a `Fiber`, runs a given callable/lambda and stores the result. Given `FiberTask` can be destroyed by a user any time, the way we handle this is to ensure that actual `FiberTask` state is owned by `FiberTaskScheduler`: if `FiberTask` is destroyed while still running, we cancel `Fiber` and free it later, once the execution is done.

Interestingly, there could be up to `N` active `Fibers` (`FiberPool` capacity), but `FiberTasks` count owned by a user can be larger. Consider next code:

```

1  FiberTask<void> m_tasks[N];
2  for (...) { m_tasks[i] = FF_async(...) }
3  // N Fiber tasks completes
4  //

```

```

5 // User still owns N m_tasks and may query
6 // the status and result - any time later.

```

Going with more optimized implementation, we could:

- preallocate N system Fibers
- preallocate a memory for up to M ($\geq N$) fiber Tasks
- each fiber Task could be limited in size (let 256 bytes); and
- each Task could free Fiber as soon as task ends

i.e., Fiber and Task lifetime could be different and everything is preallocated; meaning no allocations at runtime.

For simpler implementation, we (a) bind Fiber lifetime to a Task lifetime: Fiber is released only when Task is destroyed; and (b) Task allocates on creation, going with more standard code for type-erased implementation.

With this in mind, we start with `FiberTask_Any`:

```

1 struct FiberTask_Any : public FiberCallbackBase
2 {
3     explicit FiberTask_Any(FiberPool& fiber_pool);
4     virtual ~FiberTask_Any() noexcept;
5
6     bool is_completed() const;
7     void execute();
8     void cancel();
9
10 private:
11     virtual void do_resume() override;
12
13 protected:
14     FiberPool& _fiber_pool;
15     FiberPool::Handle _fiber;
16     bool _cancelled = false;
17 };

```

that implements `FiberCallbackBase` interface and can be cancelled, but knows nothing on how to store the result of execution:

```

1 FiberTask_Any::FiberTask_Any(FiberPool& fiber_pool)
2     : _fiber_pool(fiber_pool)
3     , _fiber(fiber_pool.allocate(*this))
4 {
5 }
6 FiberTask_Any::~FiberTask_Any() noexcept
7 {
8     assert(is_completed());

```

```

9     _fiber_pool.free(_fiber);
10 }

```

For cancellation, we throw an exception when Fiber is resumed:

```

1  class Exception_FiberTaskCancelled : public std::exception
2  {
3  };
4
5  void FiberTask_Any::cancel()
6  {
7      assert(_cancelled == false);
8      _cancelled = true;
9  }
10
11 void FiberTask_Any::do_resume()
12 {
13     if (_cancelled)
14     {
15         _cancelled = false;
16         throw Exception_FiberTaskCancelled{};
17     }
18 }

```

To execute task is to resume a Fiber:

```

1  void FiberTask_Any::execute()
2  {
3      assert(is_completed() == false);
4      _fiber_pool.resume(_fiber);
5  }

```

Finally, `is_completed()` check looks for a Fiber that ended the execution and/or has an exception thrown:

```

1  bool FiberTask_Any::is_completed() const
2  {
3      try
4      {
5          return (_fiber_pool.is_busy(_fiber) == false);
6      }
7      catch (...)
8      {
9      }
10     return true;
11 }

```

To handle void and non-void return types, we introduce `FiberTask_WithResult`:

```

1  template<typename R>
2  struct FiberTask_WithResult : public FiberTask_Any
3  {
4  public:
5      using FiberTask_Any::FiberTask_Any;
6      const R& get() const;
7      R&& get_once()
8      {
9          const R& r = static_cast<const FiberTask_WithResult&>(*this).get();
10         return std::move(const_cast<R&>(r));
11     }
12 protected:
13     // In case return type is not DefaultConstructible.
14     std::variant<std::monostate, R> _storage;
15 };
16
17 template<>
18 struct FiberTask_WithResult<void> : public FiberTask_Any
19 {
20 public:
21     using FiberTask_Any::FiberTask_Any;
22     void get() const;
23     void get_once()
24     {
25         return static_cast<const FiberTask_WithResult&>(*this).get();
26     }
27 };

```

where non-void FiberTask places the result in the `_storage` data member. `get_once()` is just a `get()` that give rvalue reference back so it could be moved from. `get()` implementation is just:

```

1  const R& get() const
2  {
3      // Populate exception, if any.
4      const bool running = _fiber_pool.is_busy(_fiber);
5      assert(running == false);
6      assert(_storage.index() == 1);
7      return std::get<1>(_storage);
8  }

```

Note how we must check Fiber's `is_busy()` to populate any exception to the user if something was throws during Fiber execution.

Finally, to run a lambda, we go with `FiberTask_Callable`:

```

1  template<typename R, typename C>
2  struct FiberTask_Callable : public FiberTask_WithResult<R>
3  {
4      using Base = FiberTask_WithResult<R>;
5  public:
6      explicit FiberTask_Callable(C&& callable, FiberPool& fiber_pool)
7          : Base(fiber_pool)
8          , _callable(std::move(callable))
9      {
10     }
11 private:
12     virtual void do_run() override
13     {
14         if constexpr (std::is_same_v<void, R>)
15         {
16             _callable();
17         }
18         else
19         {
20             this->_storage.template emplace<1>(_callable());
21         }
22     }
23 private:
24     C _callable;
25 };

```

FiberTask_Callable is what's going to be allocated on every FF_async() call that creates a FiberTask<T>:

```

1  template<typename R>
2  struct FiberTask
3  {
4      using Task = FiberTask_WithResult<R>;
5
6      template<typename C>
7      explicit FiberTask(C&& callable, FiberTaskScheduler& scheduler) noexcept
8          : _scheduler(&scheduler)
9      {
10         using TaskCallable = FiberTask_Callable<R, std::remove_cvref_t<C>>;
11         TaskCallable* task = new(std::nothrow) TaskCallable(
12             std::forward<C>(callable), scheduler._fiber_pool);
13         assert(task);
14         scheduler.add_fiber_task(std::unique_ptr<FiberTask_Any>(task));
15         _task = task;
16     }
17     ~FiberTask() noexcept

```

```

18     {
19         destroy_once();
20     }
21     void destroy_once() noexcept
22     {
23         if (_scheduler)
24         {
25             assert(_task);
26             _scheduler->remove_fiber_task(*_task);
27             _scheduler = nullptr;
28             _task = nullptr;
29         }
30     }
31     decltype(auto) get() const;
32     decltype(auto) get_once();
33     // ...
34     FiberTaskScheduler* _scheduler = nullptr;
35     Task* _task = nullptr;
36 };
37
38 template<typename C>
39 auto FF_async(FiberTaskScheduler& scheduler, C&& callable)
40 {
41     using Task = FiberTask<std::invoke_result_t<C>>;
42     return Task{std::forward<C>(callable), scheduler};
43 }

```

_task itself is owned by a FiberTaskScheduler:

```

1 struct FiberTaskScheduler
2 {
3     FiberPool& _fiber_pool;
4     std::vector<std::unique_ptr<FiberTask_Any>> _tasks;
5     std::vector<FiberTask_Any*> _tasks_to_remove;
6
7     void add_fiber_task(std::unique_ptr<FiberTask_Any>&& task)
8     {
9         assert(task.get());
10        _tasks.push_back(std::move(task));
11    }
12    void remove_fiber_task(FiberTask_Any& task)
13    {
14        if (task.is_completed() == false)
15        {
16            task.cancel();
17        }
18    }
19 }

```

```

18     _tasks_to_remove.push_back(&task);
19 }

```

Note how removing a task is also a cancel if needed.

We are left with `FiberTaskScheduler` execution, which is its `schedule()`:

```

1 void FiberTaskScheduler::schedule()
2 {
3     while (schedule_once()) {}
4 }
5
6 bool FiberTaskScheduler::schedule_once()
7 {
8     bool repeat = false;
9     auto tasks = std::move(_tasks);
10    for (auto it = tasks.rbegin(); it != tasks.rend(); ++it)
11    {
12        std::unique_ptr<FiberTask_Any>& task = *it;
13        assert(task);
14        if (task->is_completed() == false)
15        {
16            task->execute();
17            repeat |= task->is_completed();
18            continue;
19        }
20        auto it_remove = std::find(_tasks_to_remove.begin()
21                                , _tasks_to_remove.end(), task.get());
22        if (it_remove == _tasks_to_remove.end())
23        {
24            continue;
25        }
26        task.reset();
27    }
28    for (std::unique_ptr<FiberTask_Any>& task : tasks)
29    {
30        if (task)
31        {
32            _tasks.push_back(std::move(task));
33        }
34    }
35    return repeat;
36 }

```

There are 2 moments worth mentioning:

1. tasks are executed from the end - this is needed to execute child tasks first

- so parent tasks that wait can have progress
- 2. if any task is completed, we schedule a loop again - again so any parent tasks can complete if child tasks complete

Other than that, we just `.execute()` every task and remove it when completed and was removed.

Finally, running a `FiberTask` is:

```

1  int MyFiber()
2  {
3      this_fiber::suspend();
4      return 1;
5  }
6
7  int main()
8  {
9      Fiber::Boot _;
10     FiberPool fiber_pool{128};
11     FiberTaskScheduler fibers_scheduler{fiber_pool};
12
13     FiberTask<int> task = FF_async(fibers_scheduler, &MyFiber);
14     while (task.is_completed() == false)
15     {
16         fibers_scheduler.schedule();
17     }
18     std::println("{} ", task.get()); // prints "1"
19 }
```

6.7 #waiting for a CURL request with a Fiber

[source code](#).

Assuming we are withing a Fiber context already:

```

1  void Fiber_Main(CURL_Async curl_async)
2  {
3      const std::string r = CURL_fiber_get_V1(curl_async, "localhost:5001/file1.txt");
4      std::println("{} ", r);
5  }
```

Lets write our `CURL_fiber_get_V1()`:

```

1  std::string CURL_fiber_get_V1(CURL_Async curl_async, const std::string& url)
2  {
3      std::optional<std::string> response;
4      CURL_async_get(curl_async, url, &response
5          , [](void* user_data, std::string response)
6          {
```

```

7         auto& state = *static_cast<std::optional<std::string>*>(user_data);
8         state.emplace(std::move(response));
9     });
10    while (response.has_value() == false)
11    {
12        this_fiber::suspend();
13    }
14    return std::move(response.value());
15 }

```

In a way - it's trivial:

- 1) we start a GET request
- 2) we wait for a data in a while loop
- 3) while loop suspends itself if no data yet
- 4) we use a pointer to local **response** variable since it's valid until Fiber end

Note: compiler can't see or prove that local variable **response** is NOT modified externally, hence the loop is not an infinite loop. There is no need to use volatile/atomic to sidestep the compiler.

Given this - all good... Until someone CANCELS our Fiber Task. Remember, user has a reference to a FiberTask:

```

1 FiberTask<void> task = FF_async(fibers_scheduler, &Fiber_Main, curl_async);

```

What if user drops/destroys a task while GET request is still in progress?

```

1 FiberTask::~FiberTask()
2 {
3     assert(!_task);
4     _scheduler->remove_fiber_task(*_task);
5 }
6 }

```

And remove_fiber_task() just cancels the Task in progress:

```

1 void FiberTaskScheduler::remove_fiber_task(FiberTask_Any& task)
2 {
3     if (task.is_completed() == false)
4     {
5         task.cancel();
6     }
7     _tasks_to_remove.push_back(&task);
8 }

```

What happens to cancelled Fiber? There are 2 options:

- 1) we destroy a Fiber, deallocating a memory/system Fiber; and
- 2) we keep a Fiber in a pool, leaving the memory alive.

Since we have a FiberPool, cancelling a Task/Fiber does not deallocate the memory since system Fiber is still alive. So, here:

```
1  std::optional<std::string> response;
2  CURL_async_get(curl_async, url, &response
3      , [](void* user_data, std::string response)
4      {
5          auto& state = *static_cast<std::optional<std::string>*>(user_data);
6          state.emplace(std::move(response));
7      });
```

when callback is invoked and Fiber was/is cancelled, our `response` local variable is still alive and there is no access error to deallocated memory. So.. no issue?

Still, while the memory/Fiber is not deallocated, logically, it's an access to released resource and in practice, in our case - someone else could start another Fiber task that so happens gets the same Fiber. Hence, our `CURL_async_get()` callback overrides and corrupts the memory of some other task!

Sadly, cancellation is a problem at the end. To handle that - since `CURL_async_get()` is not cancellable, we need to allocate a flag/state on a heap so it can outlive cancelled Fiber:

```
1  std::string CURL_fiber_get(CURL_Async curl_async, const std::string& url)
2  {
3      std::optional<std::string> response;
4
5      CancelToken* cancel_token = CancelToken::Make(&response);
6      cancel_token->add_ref();
7
8      CURL_async_get(curl_async, url, cancel_token
9          , [](void* user_data, std::string response)
10         {
11             CancelToken& cancel_token = *static_cast<CancelToken*>(user_data);
12             if (auto* state = cancel_token.as<std::optional<std::string>>())
13             {
14                 state->emplace(std::move(response));
15             }
16             cancel_token.release();
17         });
18
19     try
20     {
21         while (response.has_value() == false)
22         {
23             this_fiber::suspend();
24         }
25         cancel_token->release();
```

```

26     }
27     catch (...) // including Exception_FiberTaskCancelled
28     {
29         cancel_token->reset();
30         cancel_token->release();
31         throw;
32     }
33
34     return std::move(response.value());
35 }

```

Logically, we:

- 1) allocate CancelToken, which is ref-counted and there are 2 users: callback and the task itself
- 2) remember a pointer to a local variable **response**
- 3) when task is cancelled (exception thrown) - we reset the reference to **response**
- 4) when callback is invoked, we check if **response** is still alive; CancelToken is guaranteed to be valid since it's ref-counted.

CancelToken implementation is just:

```

1 struct CancelToken
2 {
3     std::int64_t _ref_count = 0;
4     void* _user_data = nullptr;
5     static CancelToken* Make(void* user_data)
6     {
7         CancelToken* token = new(std::nothrow) CancelToken;
8         assert(token);
9         token->_ref_count = 1;
10        token->_user_data = user_data;
11        return token;
12    }
13    void add_ref()
14    {
15        _ref_count += 1;
16    }
17    void release()
18    {
19        _ref_count -= 1;
20        assert(_ref_count >= 0);
21        if (_ref_count == 0)
22        {
23            Destroy(this);
24        }
25    }
26 }

```

```

25     }
26     static void Destroy(CancelToken* token)
27     {
28         assert(token);
29         delete token;
30     }
31     void reset()
32     {
33         _user_data = nullptr;
34     }
35     template<typename T>
36     T* as() const
37     {
38         return static_cast<T*>(_user_data);
39     }
40 };

```

With that in mind, complete `CURL_fiber_get()` within a `FiberTask` is:

```

1 void Fiber_Main(CURL_Async curl_async)
2 {
3     const std::string r1 = CURL_fiber_get(curl_async, "localhost:5001/file1.txt");
4     const std::string r2 = CURL_fiber_get(curl_async, "localhost:5001/file2.txt");
5     std::println("{} ", r1);
6     std::println("{} ", r2);
7 }
8
9 int main()
10 {
11     Fiber::Boot _;
12     FiberPool fiber_pool{8};
13     FiberTaskScheduler fibers_scheduler{fiber_pool};
14     CURL_Async curl_async = CURL_async_create();
15     FiberTask<void> task = FF_async(fibers_scheduler, &Fiber_Main, curl_async);
16     while (task.is_completed() == false)
17     {
18         CURL_async_tick(curl_async);
19         fibers_scheduler.schedule();
20     }
21     CURL_async_destroy(curl_async);
22 }

```

Note, `FF_async()` was extended to accept extra arguments.

6.8 #waiting for multiple a FiberTasks

[source code](#).

Given a list of FiberTasks, we just wait for completion in the loop:

```
1 template<typename... Ts>
2 std::tuple<Ts...> FF_await_all(FiberTask<Ts>... tasks)
3 {
4     auto wait_task = [](auto task) -> auto
5     {
6         while (task.is_completed() == false)
7         {
8             this_fiber::suspend();
9         }
10        return task.get_once();
11    };
12    return {wait_task(std::move(tasks))...};
13 }
```

Here, we intentionally accept `tasks` by value so that all the tasks are consumed and, more importantly, if `await` is cancelled, all of the tasks are also discarded due to stack unwinding.

To be able to wait for multiple CURL requests, we need to have `FiberTask`. We do:

```
1 FiberTask<std::string> CURL_fiber_get(
2     CURL_Async curl_async
3     , const std::string& url
4     , FiberTaskScheduler& fiber_scheduler)
5 {
6     return FF_async(fiber_scheduler, [=]()
7     {
8         return CURL_fiber_get(curl_async, url);
9     });
10 }
```

Finally, doing 2 concurrent requests with Fibers is:

```
1 void Fiber_Main(FiberTaskScheduler* fiber_scheduler, CURL_Async curl_async)
2 {
3     auto [r1, r2] = FF_await_all(
4         CURL_fiber_get(curl_async, "localhost:5001/file1.txt", *fiber_scheduler),
5         CURL_fiber_get(curl_async, "localhost:5001/file2.txt", *fiber_scheduler)
6     );
7     std::println("{} ", r1);
8     std::println("{} ", r2);
9 }
```

We need `FiberTaskScheduler` to create children Fiber tasks, so it's a bit more lengthy to type all that compared with the same code for C++20 coroutines.

It's possible to get rid of `fiber_scheduler` by having it implicitly available as a global thread local variable, so `CURL_fiber_get()`/or `FF_async()` becomes:

```
1 FiberTask<std::string> CURL_fiber_get(  
2     CURL_Async curl_async  
3     , const std::string& url  
4     , FiberTaskScheduler& fiber_scheduler = FiberTaskScheduler::get_current())
```

However, that's mostly irrelevant in our context.

7 #building std::future API

[source code](#), [sample app](#).

Having `CURL_async_get()` leads to the next implementation that wraps everything into a `std::future`:

```
1 std::future<std::string> CURL_future_get(  
2     CURL_Async curl_async, const std::string& url)  
3 {  
4     using Promise = std::promise<std::string>;  
5     Promise* promise = new(std::nothrow) Promise;  
6     assert(promise);  
7     std::future<std::string> future = promise->get_future();  
8     CURL_async_get(curl_async, url, promise  
9         , [](void* user_data, std::string response)  
10    {  
11        Promise* promise = static_cast<Promise*>(user_data);  
12        promise->set_value(std::move(response));  
13        delete promise;  
14    });  
15     return future;  
16 }
```

Everything is just what `std::` exposes. Note how we need to allocate our promise so it can be alive until the end of the request.

All in all, we:

1. allocate `std::promise`
2. `std::promise` allocates shared state, used by `std::future`
3. `CURL_async_get` allocates `std::string` to write a response
4. `CURL_async_get` allocates `std::function` for a generic callback
5. `CURL_async_get` allocates `std::unordered_map` node to remember what to call when
6. .. and probably something else (CURL internals, etc)

Anyway, `std::future` is missing useful bits to work with it in a non-blocking manner:

- no built-in `.is_ready()` check
- no built-in `.then()` continuation, see [Design and evolution of C++ future continuations](#)

Faking `is_future_ready()` with:

```
1 template<typename T>
2 bool is_future_ready(const std::future<T>& future)
3 {
4     const std::future_status status = future.wait_for(std::chrono::seconds::zero());
5     return (status == std::future_status::ready);
6 }
```

allows to finally write something among the lines:

```
1 int main()
2 {
3     CURL_Async curl_async = CURL_async_create();
4     std::future<std::string> result = CURL_future_get(
5         curl_async, "localhost:5001/file1.txt");
6     while (is_future_ready(result) == false)
7     {
8         CURL_async_tick(curl_async);
9     }
10    CURL_async_destroy(curl_async);
11    std::println("{} ", result.get());
12 }
```

Missing `std::future` features makes it not composable; waiting 2 tasks to finish is the same as checking 2 flags to become true; giving not much of a win compared to direct use of `CURL_async_get()`.

8 #building task API with `.then()` support

[source code](#), [sample app](#).

Lets build `std::future`-like `Task` type that supports `.then()` continuations.

Everything is the same as `std::future`, but there is a support to chain operations, see [std::experimental::future::then\(\)](#) and [boost::future::then\(\)](#):

```
1 Task<std::string> CURL_task_get(CURL_Async curl_async, const std::string& url);
2
3 int main()
4 {
5     Task<void> task = CURL_task_get(curl_async, "localhost:5001/file1.txt")
```

```

6         .then([](std::string response)
7         {
8             std::println("{} ", response);
9         });
10        // ...
11    }

```

`.then()` returns new `Task` that holds the value of whatever our callable returned (`Task<void>` in the example above), but it can be a value:

```

1    Task<std::string> t;
2    Task<int> next = t.then([](std::string)
3    {
4        return 573;
5    });

```

With such an API, `.then()` needs to return a `Task` to the caller while also holding same task data to set the value to it once previous task completes. That requires to have a separate reference-counted block under the hood. Lets start with a simple `Task_RefCount` then:

```

1    struct Task_RefCount
2    {
3        std::int64_t _ref_count = 1;
4        Task_RefCount() noexcept = default;
5        Task_RefCount(const Task_RefCount&) = delete;
6
7        void add_ref()
8        {
9            assert(_ref_count >= 0);
10           _ref_count += 1;
11        }
12
13        bool release()
14        {
15            assert(_ref_count >= 1);
16            _ref_count -= 1;
17            if (_ref_count == 0)
18            {
19                Deallocate(this);
20                return true;
21            }
22            return false;
23        }
24
25        static void Deallocate(Task_RefCount* ptr)
26        {

```

```

27         assert(ptr);
28         delete ptr;
29     }
30
31     virtual ~Task_RefCount() = default;
32 };

```

We also re-use `std::unique_ptr` to hold the value of `Task_RefCount`, so there is no need to manually implement move operations:

```

1 struct Task_Release
2 {
3     void operator()(Task_RefCount* ptr) noexcept
4     {
5         assert(ptr);
6         ptr->release();
7     }
8 };
9
10 template<typename T>
11 using Task_Ptr = std::unique_ptr<T, Task_Release>;

```

Now, our `Task` holds any value `T`. Lets implement `Task_Storage` that knows how to handle that since there are at least 2 complications:

- 1) `T` can be void and `T` can be a reference; those can't be directly stored and
- 2) `T` can be not default constructible

```

1 template<typename T>
2 struct Task_Storage : Task_RefCount
3 {
4     std::variant<std::monostate, T> _value;
5     template<typename U>
6     void set_value(U&& v) noexcept
7     {
8         assert(has_value() == false);
9         _value.template emplace<1>(std::forward<U>(v));
10        finish();
11    }
12    bool has_value() const
13    {
14        return (_value.index() == 1);
15    }
16    const T& get() const noexcept
17    {
18        assert(has_value());
19        return std::get<1>(_value);

```

```

20     }
21     T consume() noexcept
22     {
23         assert(has_value());
24         T v{std::move(std::get<1>(_value))};
25         _value.template emplace<0>();
26         return v;
27     }
28     template<typename F>
29     decltype(auto) dispatch_once(F&& f)
30     {
31         return std::invoke(std::forward<F>(f), consume());
32     }
33     virtual void finish() = 0;
34 };

```

`std::variant<std::monostate, T>` is there to solve non-default-constructible case (could be `std::optional<T>`, I prefer variant) and `T&` and `void` cases are `Task_Storage` specializations:

```

1  template<typename T>
2  struct Task_Storage<T& > : Task_RefCount
3  {
4      T* _value = nullptr;
5  };
6
7  template<>
8  struct Task_Storage<void> : Task_RefCount
9  {
10     bool _has_value = false;
11 };

```

The code for those is largely the same as our base case. While `set_value()`, `has_value()` and `get()` are self-explanatory, we also have:

- `T consume()`: just a `get()` that move the value out of the storage;
- `dispatch_once(F)`: calls the callable `F` with our value; mostly needed to specialize void case so `F` could be invoked with no arguments; and
- pure virtual `finish()` that is invoked when we `set_value()`: so we can handle `.then()` implementation/completion/continuation.

Strictly speaking, that's all we need to implement just `Task<T>`. Still, for `.then(F)`, we need to store any user-defined callable `F`. Hence, we extend `Task_Storage` with `Task_Callback`:

```

1  template<typename T>
2  struct Task_Callback : Task_Storage<T>
3  {

```

```

4     std::move_only_function<void ()> _callback;
5     virtual void finish() override
6     {
7         if (_callback)
8         {
9             std::move_only_function<void ()> call = std::move(_callback);
10            call();
11        }
12    }
13 };

```

where the “trick” is to use `std::function` to handle all of/any user-defined callable. So, at the end, when we `set_value()` on our `Task`, this callback is invoked once (if set). We’ll use that for `.then` implementation, but, for now, let’s see all of `Task` details where we simply allocate `Task_Callback`:

```

1  template<typename T>
2  struct Task
3  {
4      using type = T;
5      Task_Ptr<Task_Callback<T>> _ptr;
6      explicit Task() noexcept
7          : _ptr(new(std::nothrow) Task_Callback<T>{})
8      {
9          assert(_ptr.get());
10     }
11     ~Task() noexcept = default;
12     Task(Task&& rhs) noexcept = default;
13     Task& operator=(Task&& rhs) noexcept = default;
14     Task(const Task&) = delete;
15     Task& operator=(const Task&) = delete;

```

Then implement the usual getters/setters (`set_value()` and `get()` handles void case too):

```

1     decltype(auto) get() const
2     {
3         assert(is_valid());
4         return _ptr->get();
5     }
6
7     decltype(auto) get_once()
8     {
9         assert(is_valid());
10        return _ptr->consume();
11    }
12

```

```

13     template<typename... Ts>
14         requires ((sizeof...(Ts)) == 0)
15     void set_value(Ts&&... vs)
16     {
17         assert(is_valid());
18         _ptr->set_value();
19     }
20     template<typename... Ts>
21         requires ((sizeof...(Ts)) == 1)
22     void set_value(Ts&&... vs)
23     {
24         assert(is_valid());
25         (_ptr->set_value(std::forward<Ts>(vs)), ...);
26     }
27
28     bool has_value() const
29     {
30         assert(is_valid());
31         return _ptr->has_value();
32     }
33
34     bool is_valid() const
35     {
36         return (_ptr.get() != nullptr);
37     }

```

std::future does not expose set_value on a future directly to limit the number of API misuse, std::promise needs to be used; but we go with a simpler code. Finally, to expose reference-counter task under the hood, we add **share()** API:

```

1  explicit Task(Task_Callback<T>* ptr) noexcept // explicit share
2      : _ptr(ptr)
3  {
4      assert(ptr);
5      ptr->add_ref();
6  }
7  Task share()
8  {
9      assert(is_valid());
10     return Task(_ptr.get());
11 }

```

which allows to give a Task to the user, but also have a reference to it to set the value later, so we can implement .then(), which roughly does:

```

1  auto Task<T>::then(F&& f)
2  {

```

```

3     using Return = invoke_result_t<F, T>;
4     Task<Return> task;
5     _ptr->_callback = [target = task.share()]() // **HERE: remember task
6     {
7         target.set_value(...);
8     }
9     return task;
10 }

```

There is one complication to implement `.then()` that easily: implicit task unwrap. Going back to `.then()` example:

```

1 Task<std::string> t;
2 Task<int> next = t.then([](std::string)
3 {
4     return 573;
5 });

```

when our callable returns `int`, `.then()` returns `Task<int>`. What if we want to start another task within the callback?:

```

1 Task<std::string> t;
2 Task<??> next = t.then([](std::string)
3 {
4     return CURL_task_get("...");
5 });

```

`CURL_task_get()` returns `Task<std::string>` on its own. This chaining is so common that instead of just returning `Task<Task<std::string>>` to the user, we want to return `Task<std::string>` directly that would represent the completed result of the inner `CURL_task_get()` operation. Hence, our `.then()` implementation checks the return type of the callable:

```

1 template<typename T>
2 template<typename F>
3 auto Task<T>::then(F&& f)
4 {
5     using Return = invoke_result_t<F, T>;
6     if constexpr (is_task_type_v<Return>)
7     {
8         Task<typename Return::type> task;
9         attach_callback(task, std::forward<F>(f));
10        return task;
11    }
12    else
13    {
14        Task<Return> task;
15        attach_callback(task, std::forward<F>(f));

```

```

16         return task;
17     }
18 }

```

and if the callable `Return=Task<U>`, we get `U` out of it and return `Task<U>` to the user; otherwise, it's just `Task<Return>` as it is. The helper `this->attach_callback(target, callback)`:

- waits for task finish
- then invokes callback
- then sets the value of that callback to the target task

Lets see the simple case:

```

1 void Task<T>::attach_callback(Task<U>& target, F&& callback)
2 {
3     _ptr->_callback = [
4         f = std::forward<F>(callback)
5         , self_ptr = _ptr.get()
6         , target = target.share()
7     ]() mutable
8     {
9         target.set_value(
10             self_ptr->dispatch_once(std::move(f))); // calls f
11     };
12
13     if (_ptr->has_value())
14     {
15         _ptr->finish(); // invokes _callback
16     }
17     // else: to be invoked later, on a first call to set_value()
18 };

```

where we:

- 1) remember the `target` task for which the value needs to be set once we are done;
- 2) setup `_callback` that's going to be executed on our `finish()`;
- 3) once on `finish()`, we invoke the callback with `dispatch_once()` wrapper; and
- 4) set a value for our target task.

Note that `target` task IS what user gets (here `target` is our `next`):

```

1 Task<std::string> t;
2 Task<int> next = t.then([](std::string)
3 {
4     return 573;
5 });

```

On the other hand, if user callable returns `Task<T>`, we need to attach callback to that Task instead:

```

1 void Task<T>::attach_callback(Task<U>& target, F&& callback)
2 {
3     using Result = invoke_result_t<F, T>;
4
5     _ptr->_callback = [
6         f = std::forward<F>(callback)
7         , self_ptr = _ptr.get()
8         , target = target.share()
9     ]() mutable
10    {
11        if constexpr (is_task_type_v<Result> == false)
12        {
13            target.set_value(
14                self_ptr->dispatch_once(std::move(f)));
15        }
16        else // unwrap inner Task
17        {
18            auto inner_task = self_ptr->dispatch_once(std::move(f));
19            inner_task.attach_callback(target, Identity_Callback<U>{});
20        }
21    };
22
23    if (_ptr->has_value())
24    {
25        _ptr->finish();
26    }
27 };

```

There are nuances of handling the case when callable returns `void` (see [source code](#)).

With that in mind, we can use Tasks like this:

```

1 Task<int> task;
2 task.then([](int v)
3 {
4     std::println("got {}", v);
5 });
6 std::println("-- set task to 5");
7 task.set_value(5); // invokes callback

```

or unwrap the inner task like so:

```

1 Task<void> task;
2 Task<int> inner;

```

```

3 Task<int> end = task.then([&inner]()
4 {
5     std::println("task end");
6     return inner.share();
7 });
8 std::println("-- set task");
9 task.set_value();
10 assert(end.has_value() == false); // not yet, inner is in progress
11 std::println("-- set INNER task to 9");
12 inner.set_value(9);
13 std::println("got {}", end.get()); // prints 9

```

That task chaining is less unintuitive once real function that returns Task is invoked instead of dummy inner task. Lets wrap CURL_async_get() to return a Task.

8.1 #wrapping CURL get into a Task

[source code](#).

Implementing CURL_task_get() is trivial:

```

1 Task<std::string> CURL_task_get(CURL_Async curl_async, const std::string& url)
2 {
3     Task<std::string> task;
4     void* ptr = task.share_address();
5     CURL_async_get(curl_async, url, ptr
6         , [](void* user_data, std::string response)
7     {
8         Task<std::string> task = Task<std::string>::from_address(user_data);
9         task.set_value(std::move(response));
10    });
11    return task;
12 }

```

Here, to pass void* pointer to our CURL_async_get(), we implement share_address():

```

1 void* share_address()
2 {
3     assert(_ptr);
4     _ptr->add_ref();
5     return _ptr.get();
6 }
7
8 static Task from_address(void* ptr)
9 {

```

```

10     assert(ptr);
11     Task_Callback<T>* task_ptr = static_cast<Task_Callback<T>*>(ptr);
12     Task task{task_ptr};
13     task_ptr->release();
14     return task;
15 }

```

That's possible since our internal pointer is reference counted and we have full access to the details.

Doing 2 sequential CURL get requests is

```

1  static Task<void> Main_Task(CURL_Async curl_async)
2  {
3      return CURL_task_get(curl_async, "localhost:5001/file1.txt")
4          .then([curl_async](std::string r1)
5          {
6              std::println("{} ", r1);
7              return CURL_task_get(curl_async, "localhost:5001/file2.txt")
8                  .then([](std::string r2)
9                  {
10                     std::println("{} ", r2);
11                 });
12          });
13  }
14
15  int main()
16  {
17      CURL_Async curl_async = CURL_async_create();
18      Task<void> task = Main_Task(curl_async);
19      while (task.has_value() == false)
20      {
21          CURL_async_tick(curl_async);
22      }
23      CURL_async_destroy(curl_async);
24  }

```

8.2 #waiting for multiple Task requests

[source code](#).

Lets wait for N Tasks - Tasks_WhenAll():

```

1  template<typename... Ts>
2  Task<std::tuple<Ts...>> Tasks_WhenAll(Task<Ts>&&... tasks);
3
4  int main()

```

```

5  {
6      Task<int> t1;
7      Task<char> t2;
8      Tasks_WhenAll(t1.share(), t2.share())
9          .then([](std::tuple<int, char> v)
10         {
11             auto [x, y] = v;
12             std::println("{} {}", x, y);
13         });
14     t1.set_value(1);
15     t2.set_value('a');
16 }

```

Given 2 tasks, Task<int> and Task<char>, waiting for them give us std::tuple<int, char> back. Since tasks may be not ready yet, we return Task<std::tuple<int, char>>. Conceptually, we need to do something like this:

```

1  Task<std::tuple<int, char>> Tasks_WhenAll(Task<int> t1, Task<char> t2)
2  {
3      Task<std::tuple<int, char>> out;
4      t1.then([](int v)
5      {
6          // set v to slot 0 of out tuple
7          // complete out task if we are last
8      });
9      t2.then([](char v)
10     {
11         // set v to slot 1 of out tuple
12         // complete out task if we are last
13     });
14     return out;
15 }

```

Again, since t1 and t2 may complete out of order, we need to resort to reference counting again and have some internal state that completes when last task completes.

Lets re-use Task_Storage that is reference-counted already. For a case of waiting for Task<int> and Task<char>, we have:

```

1  auto Tasks_WhenAll0(Task<int>&& task0, Task<char>&& task1)
2  {
3      auto* state = new(std::nothrow) WhenAll_State0{};
4      assert(state);
5
6      state->start_all(std::move(task0), std::move(task1));

```

```

7
8     auto task = state->_task.share();
9     state->release(); // already owned by start_all()
10    return task;
11 }

```

where `state` manages all the logic. WhenAll_State0 example is:

```

1 struct WhenAll_State0 : Task_Storage<std::tuple<int, char>>
2 {
3     Task<std::tuple<int, char>> _task;
4     WhenAll_State0() noexcept
5     {
6         this->_value.template emplace<1>();
7     }
8     ~WhenAll_State0() noexcept
9     {
10        finish();
11    }
12    std::tuple<int, char>& data()
13    {
14        return std::get<1>(this->_value);
15    }
16    virtual void finish() override
17    {
18        _task.set_value(this->consume());
19    }

```

where we just use `Task_Storage` as our implementation detail and on `finish()` - move the data from out internal storage to the task itself. `start_all()` starts waiting for both of the tasks:

```

1 void WhenAll_State0::start_all(Task<int>&& task0, Task<char>&& task1)
2 {
3     start0(std::move(task0));
4     start1(std::move(task1));
5 }
6 void WhenAll_State0::start0(Task<int>&& task)
7 {
8     this->add_ref();
9     task.then([this, self = Task_Ptr<>{this}](int v)
10    {
11        std::get<0>(data()) = std::move(v);
12    });
13 }
14 void WhenAll_State0::start1(Task<char>&& task)
15 {

```

```

16     this->add_ref();
17     task.then([this, self = Task_Ptr<>{this}](char v)
18     {
19         std::get<1>(data()) = std::move(v);
20     });
21 }

```

start0() does:

1. increment our reference count so this state is alive until the end of the task
2. remember itself as `self = Task_Ptr<>{this}` to properly decrement the counter
3. on complete, save the value AND decrement `self` (implicitly destroyed).

More generic implementation of `Tasks_WhenAll()` does the same:

```

1  template<typename... Ts>
2  auto Tasks_WhenAll(Task<Ts>&&... tasks)
3  {
4      static_assert(sizeof...(Ts) >= 1);
5      static_assert(std::conjunction_v<is_value_type<Ts>...>
6                    , "Task<void> or Task<T&> is not implemented");
7      static_assert(std::conjunction_v<std::is_default_constructible<Ts> ...>
8                    , "Waiting a Task<T> with non-default-constructible T is not implemented");
9
10     using Tuple = std::tuple<Ts...>;
11
12     auto* state = new(std::nothrow) WhenAll_State<Task<Tuple>>{};
13     assert(state);
14
15     state->start_all(std::tuple<Task<Ts>...>{std::move(tasks)...}
16                   , std::index_sequence_for<Ts...>{});
17
18     auto task = state->_task.share();
19     state->release();
20     return task;
21 }

```

The limitations (no `Task<void>` or `Task<T&>`) could be implemented, but do not change the point.

9 #building C++26 senders

[source code](#), [sample app](#).

Required read: [What are Senders Good For, Anyway?](#).

`std::execution`, accepted in C++26 [P2300R10](#) provides a framework for managing

asynchronous execution. Eric Niebler has a nice intro above among few other [videos](#) on the topic. Some examples of senders and receivers are provided in [P2300R10](#). In addition, stdexec has nice [Developer's Guide](#).

Ultimately, we'd like to write a Sender that wraps our `CURL_async_get()` and produces a response:

```
1 SENDER CURL_sender_get(CURL_Async curl_async, const std::string& url);
2
3 auto App_Senders(CURL_Async curl_async)
4 {
5     return CURL_sender_get(curl_async, "localhost:5001/file1.txt")
6         | stdexec::then([](std::string r)
7         {
8             std::println("{} ", r);
9         });
10 }
```

or, with Sender's coroutines support:

```
1 stdexec::task<void> App_Senders(CURL_Async curl_async)
2 {
3     const std::string r = co_await CURL_sender_get(
4         curl_async, "localhost:5001/file1.txt");
5     std::println("{} ", r);
6 }
```

We'll use `stdexec` for a start. Start from [senders basics](#) for a simplified senders implementation.

As of [2026/09/13](#), stdexec requires at least Visual Studio 2022 version 17.13.0 (MSVC 14.43).

We start by building simplest sender that does nothing:

```
1 struct Sender
2 {
3     // ...
4 };
5
6 Sender CURL_get()
7 {
8     return Sender{};
9 }
10
11 int main()
12 {
13     stdexec::sync_wait(CURL_get());
14 }
```

where `sync_wait()` starts our Sender and blocks the execution until the end.
To make it compile, `Sender` is missing several required bits:

```
1 struct Sender
2 {
3     using sender_concept = stdexec::sender_tag;
4     using completion_signatures = stdexec::completion_signatures<
5         stdexec::set_value_t ()>;
6
7     template<typename Receiver>
8     auto connect(Receiver&& receiver)
9     {
10         using Receiver_ = std::remove_cvref_t<Receiver>;
11         return State<Receiver_>{std::forward<Receiver>(receiver)};
12     }
13 };
```

where we, basically, say that:

- 1) Sender is... a Sender concept (`sender_tag`).
- 2) Our Sender will only return void/nothing - that `completion_signatures`.
And
- 3) The Sender requires State when connected to any compatible Receiver.

(If nothing clicks, read [What are Senders Good For, Anyway?](#)).

Our State is:

```
1 template<typename Receiver>
2 struct State
3 {
4     using operation_state_concept = stdexec::operation_state_tag;
5
6     void start() noexcept
7     {
8         stdexec::set_value(std::move(_receiver));
9     }
10
11     Receiver _receiver;
12 };
```

where we:

- 1) Say that State is `operation_state` concept.
- 2) When operation starts, we immediately complete it by invoking `set_value()` on our receiver.

With this, we can use our Sender:

```

1  Sender CURL_get()
2  {
3      return Sender{};
4  }
5
6  int main()
7  {
8      std::optional<std::tuple<>> x = stdexec::sync_wait(CURL_get());
9      assert(x.has_value());
10 }

```

Note: `sync_wait()` returns an optional since, in general, sender/operation can fail and a tuple, since sender can return multiple values.

Since Sender concept is compatible with C++20 coroutines awaitable, just implementing a Sender allows to use it in coroutines. So, next coroutine works just fine too:

```

1  Sender CURL_get()
2  {
3      return Sender{};
4  }
5
6  stdexec::task<void> CURL_get_coro()
7  {
8      co_await CURL_get();
9  }
10
11 int main()
12 {
13     std::optional<std::tuple<>> x = stdexec::sync_wait(CURL_get_coro());
14     assert(x.has_value());
15 }

```

9.1 #sender for a CURL get

[source code](#).

Lets adapt our simple Sender to return (“send”) a `std::string` - i.e., what `CURL_async_get()` returns:

```

1  CURL_Get_Sender CURL_sender_get(CURL_Async curl_async, const std::string& url)
2  {
3      return CURL_Get_Sender{}; // TBD
4  }

```

where `CURL_Get_Sender` is:

```

1  template<typename Receiver>
2  struct CURL_Get_State
3  {
4      using operation_state_concept = stdexec::operation_state_tag;
5
6      void start() noexcept
7      {
8          stdexec::set_value(std::move(_receiver), std::string{});
9      }
10
11     Receiver _receiver;
12 };
13
14 struct CURL_Get_Sender
15 {
16     using sender_concept = stdexec::sender_tag;
17     using completion_signatures = stdexec::completion_signatures<
18         stdexec::set_value_t (std::string)>;
19
20     template<typename Receiver>
21     auto connect(Receiver&& receiver)
22     {
23         using Receiver_ = std::remove_cvref_t<Receiver>;
24         return CURL_Get_State<Receiver_>{std::forward<Receiver>(receiver)};
25     }
26 };

```

See, completion_signatures now indicate that we set_value(std::string) and inside operation start() we now pass an empty std::string.

Can we use our dummy CURL_sender_get() already?

```

1  int main()
2  {
3      CURL_Async curl_async = CURL_async_create();
4      std::optional<std::tuple<std::string>> x =
5          stdexec::sync_wait(CURL_sender_get(curl_async, "localhost:5001/file1.txt"));
6      assert(x.has_value());
7      while (???)
8      {
9          CURL_async_tick(curl_async);
10     }
11     CURL_async_destroy(curl_async);
12 }

```

Yes. But note there are 2 issues:

- 1) `sync_wait()` will block execution and we'll never tick our CURL loop;
(note: we probably could inject our tick logic into `sync_wait` `run_loop`);
- 2) we don't know when to stop.

To continue `main()` execution, we must remove `sync_wait` and manually start the Sender. To do that, we wrap all and every senders logic in `Senders_Main()`:

```

1 auto Senders_Main(CURL_Async curl_async)
2 {
3     return CURL_sender_get(curl_async, "localhost:5001/file1.txt")
4         | stdexec::then([](std::string r)
5         {
6             std::println("{} ", r);
7         });
8 }
```

This is where anything/everything related to senders should be done. Next, we can manually `connect()` and `start()` our `Senders_Main()`:

```

1 int main()
2 {
3     CURL_Async curl_async = CURL_async_create();
4     bool done = false;
5     auto state = stdexec::connect(Senders_Main(curl_async), AnyReceiver{&done});
6     stdexec::start(state);
7     while (done == false)
8     {
9         CURL_async_tick(curl_async);
10    }
11    CURL_async_destroy(curl_async);
12 }
```

where `AnyReceiver` is a generic receiver that accepts any and all values and just sets true to a bool flag:

```

1 struct AnyReceiver
2 {
3     using receiver_concept = stdexec::receiver_tag;
4
5     template<typename... Args>
6     void set_value(Args&&...) noexcept { finish(); }
7     template<typename... Args>
8     void set_error(Args&&...) noexcept { finish(); }
9     void set_stopped() noexcept { finish(); }
10
11     void finish() noexcept
12     {
13         assert(_done);
```

```

14         assert(*_done == false);
15         *_done = true;
16     }
17
18     bool* _done = nullptr;
19 };

```

Having a `main()` that properly runs CURL main loop, we can go back to `CURL_sender_get()` implementation:

```

1 struct CURL_Get_Sender
2 {
3     using sender_concept = stdexec::sender_tag;
4     using completion_signatures = stdexec::completion_signatures<
5         stdexec::set_value_t (std::string)>;
6
7     CURL_Async _curl_async = nullptr;
8     std::string _url;
9     // ...
10 };
11
12 CURL_Get_Sender CURL_sender_get(CURL_Async curl_async, const std::string& url)
13 {
14     return CURL_Get_Sender
15     {
16         ._curl_async = curl_async,
17         ._url = url
18     };
19 }

```

See how `CURL_sender_get()` does not do any work and passes any required input to `CURL_Get_Sender` itself. `CURL_Get_Sender` remembers everything until `connect()` is invoked. This is where operation state is finally constructed:

```

1 struct CURL_Get_Sender
2 {
3     CURL_Async _curl_async = nullptr;
4     std::string _url;
5
6     template<typename Receiver>
7     auto connect(Receiver&& receiver)
8     {
9         using Receiver_ = std::remove_cvref_t<Receiver>;
10        return CURL_Get_State<Receiver_>
11        {
12            ._receiver = std::forward<Receiver>(receiver),
13            ._curl_async = _curl_async,

```

```

14         ._url = std::move(_url)
15     };
16 }
17 };
18
19 template<typename Receiver>
20 struct CURL_Get_State
21 {
22     void start() noexcept;
23
24     Receiver _receiver;
25     CURL_Async _curl_async = nullptr;
26     std::string _url;
27 };

```

CURL_Get_State also waits for start() to be invoked. At that point, we:

- 1) can start the actual operation/work; and
- 2) our state is guaranteed to be alive until we end it on our side

meaning, that we can pass a pointer to state around and it's guaranteed to be alive:

```

1 void CURL_Get_State::start() noexcept
2 {
3     assert(_curl_async);
4     CURL_async_get(_curl_async, _url
5         , this // HERE, pointer to our state
6         , [](void* user_data, std::string response)
7     {
8         CURL_Get_State& state = *static_cast<CURL_Get_State*>(user_data);
9         stdexec::set_value(std::move(state._receiver), std::move(response));
10    });
11 }

```

Once CURL_async_get() callback is invoked, we simply invoke the receiver and pass the retrieved response as a result.

One more time, full CURL async get sender implementation is:

```

1 template<typename Receiver>
2 struct CURL_Get_State
3 {
4     using operation_state_concept = stdexec::operation_state_tag;
5
6     void start() noexcept
7     {
8         assert(_curl_async);

```

```

9         CURL_async_get(_curl_async, _url
10             , this
11             , [](void* user_data, std::string response)
12             {
13                 CURL_Get_State& state = *static_cast<CURL_Get_State*>(user_data);
14                 stdexec::set_value(std::move(state._receiver), std::move(response));
15             });
16     }
17
18     Receiver _receiver;
19     CURL_Async _curl_async = nullptr;
20     std::string _url;
21 };
22
23 struct CURL_Get_Sender
24 {
25     using sender_concept = stdexec::sender_tag;
26     using completion_signatures = stdexec::completion_signatures<
27         stdexec::set_value_t (std::string)>;
28
29     CURL_Async _curl_async = nullptr;
30     std::string _url;
31
32     template<typename Receiver>
33     auto connect(Receiver&& receiver)
34     {
35         using Receiver_ = std::remove_cvref_t<Receiver>;
36         return CURL_Get_State<Receiver_>
37         {
38             ._receiver = std::forward<Receiver>(receiver),
39             ._curl_async = _curl_async,
40             ._url = std::move(_url)
41         };
42     }
43 };
44
45 CURL_Get_Sender CURL_sender_get(CURL_Async curl_async, const std::string& url)
46 {
47     return CURL_Get_Sender
48     {
49         ._curl_async = curl_async,
50         ._url = url
51     };
52 }

```

That allows to use existing, composable senders algorithms. Executing 2 GET requests in sequence now becomes:

```
1 auto Senders_Main(CURL_Async curl_async)
2 {
3     return exec::sequence(
4         CURL_sender_get(curl_async, "localhost:5001/file1.txt")
5         | stdexec::then([](std::string r1)
6         {
7             std::println("{} ", r1);
8         }),
9         CURL_sender_get(curl_async, "localhost:5001/file2.txt")
10        | stdexec::then([](std::string r2)
11        {
12            std::println("{} ", r2);
13        }));
14 }
```

9.2 #senders basics: implementing then() and sync_wait()

[source code](#).

std::execution (with stdexec) is complex, generic and handles wide range of cases.

Going with few assumptions and restrictions allows to show the core idea behind and implement basic version of senders and receivers. We'll assume:

- sender can only send one value T; so we have only set_value(T)
- sender can only fail with one value E; so we have only set_error(E)
- value and error is non-void type; so we don't need to branch void case
- no customization points
- no exceptions, everything is noexcept (including user-defined lambdas)
- we skip advanced concepts, like domains, environments and cancellation (see [P2300R10](#))

With that, we can start with coding the basic ideas:

```
1 // Receiver.
2 template<typename Receiver, typename T>
3 void set_value(Receiver&& r, T&& v) noexcept
4 {
5     FWD(r).set_value(FWD(v));
6 }
7
8 template<typename Receiver, typename E>
9 void set_error(Receiver&& r, E&& e) noexcept
10 {
11     FWD(r).set_error(FWD(e));
```

```

12 }
13
14 template<typename Receiver>
15 void set_stopped(Receiver&& r) noexcept
16 {
17     FWD(r).set_stopped();
18 }

```

See, everything we can do with Receiver is those 3 things: `set_value(T)`, `set_error(E)` and `set_stopped()`.

A note on FWD: typing `std::forward<Receiver>(r)` clutters the details; we simplify:

```

1 #define FWD(...) ::std::forward<decltype(__VA_ARGS__)>(__VA_ARGS__)
2 #define MOV(...) ::std::move(__VA_ARGS__)
3 #define REMOVE_CVR(...) std::remove_cvref_t<__VA_ARGS__>
4 using void_t = std::monostate;

```

For Sender, we define next API:

```

1 // Sender.
2 template<typename Sender, typename Receiver>
3 auto connect(Sender&& s, Receiver&& r) noexcept
4 {
5     return FWD(s).connect(FWD(r));
6 }
7
8 template<typename Sender>
9 using sender_value_t = typename REMOVE_CVR(Sender)::value_t;
10
11 template<typename Sender>
12 using sender_error_t = typename REMOVE_CVR(Sender)::error_t;

```

Again, for Sender, we can only (a) `connect()` it to a Receiver and (b) query the types we would eventually send.

Finishing with Operation API, we can only `start()`:

```

1 // Operation.
2 template<typename Operation>
3 void start(Operation& o) noexcept
4 {
5     o.start();
6 }

```

With only the pieces above, we can implement `just(v)` Sender:

```

1  template<typename T>
2  auto just(T&& v) noexcept
3  {
4      return Sender_Just<REMOVE_CVR(T)>{._v = FWD(v)};
5  }

```

so... just() only returns a wrapper (Sender_Just) that remembers a value v we would set later. No actual work done or started. Sender_Just is:

```

1  template<typename T>
2  struct Sender_Just
3  {
4      using value_t = T;
5      using error_t = void_t;
6      T _v;
7      template<typename Receiver>
8      auto connect(Receiver&& r) noexcept
9      {
10         return State_Just<REMOVE_CVR(Receiver), T>{._r = FWD(r), ._v = MOV(_v)};
11     }
12 };

```

where we signal that our operation would set _value() of a type T; there is no error; and when just Sender is connected to a Receiver, we... return the proper state we need; again, no actual work is done:

```

1  template<typename Receiver, typename T>
2  struct State_Just
3  {
4      Receiver _r;
5      T _v;
6      void start() noexcept
7      {
8          set_value(MOV(_r), MOV(_v));
9      }
10 };

```

Operation state (State_Just) is the non-movable, always alive (during operation) state that knows how to start the operation. For a just(1) case, we complete the operation immediately - by invoking set_value() on a Receiver and passing the value we have.

One more time - Receiver could be thought as as callable/lambda. Saying:

invoking set_value() on a Receiver [...]

is just a fancy way to say that we call a lambda and pass a result value to it.

For now, we can't use a lambda as a Receiver directly (then() needs to be

implemented). For a test, lets have a simple one:

```
1 struct LogReceiver
2 {
3     void set_value(auto&& v) noexcept
4     {
5         std::println("set_value({})", v);
6     }
7 };
8
9 int main()
10 {
11     auto operation_state = connect(just(399), LogReceiver{});
12     start(operation_state); // *
13 }
```

So:

- we have a just Sender
- we create a just Sender instance by calling `just(399)`
- `just(399)` describes a work that would be done when operation starts
- we connect the Sender to a specific instance of a Receiver (read “callback”)
- connecting the Sender and the Receiver gives us Operation state back
- Operation state encodes all that work and data that are needed to execute everything; finally
- we start an operation by invoking a `.start()` on an operation state

We do not wait for operation completion in the code above because we know everything completes immediately, inline and we see the output:

```
1 set_value(399)
```

`sync_wait` does mostly the same under the hood. Lets implement it. `sync_wait()` accepts any Receiver, starts it, waits for it and returns the result:

```
1 int main()
2 {
3     auto x = sync_wait(just(1));
4 }
```

Since any Sender, in general, can return either value T on success, error E on error or cancel signal (`.set_stopped()`), lets first write a type that can represent that:

```
1 template<typename T, typename E>
2 struct sync_wait_result : std::variant<std::monostate, T, E>
3 {
4     bool was_stopped() const { return (this->index() == 0); }
5     bool has_value() const { return (this->index() == 1); }
6     bool has_error() const { return (this->index() == 2); }
```

```

7     T& value() { return std::get<1>(*this); }
8     E& error() { return std::get<2>(*this); }
9 };

```

(Real `std::exec::sync_wait()` does it differently, mostly because errors are handled differently).

Now, we can:

```

1  template<typename Sender>
2  auto sync_wait(Sender&& s) noexcept
3  {
4      using T = sender_value_t<Sender>;
5      using E = sender_error_t<Sender>;
6      State_SyncWait<T, E> state;
7      auto o = connect(FWD(s), Receiver_SyncWait<T, E>{._state = &state});
8      start(o);
9      state.wait();
10     return MOV(state.get());
11 }

```

See, given a Sender, we:

- query its value type (for success) and error type
- create our internal wait state (`State_SyncWait`)
- (note it's all local variable on the stack since we implement blocking wait); then
- connect a Sender with our Receiver (that knows how to notify operation end); then
- start an operation; finally
- do a blocking wait

Lets check what `Receiver_SyncWait` does:

```

1  template<typename T, typename E>
2  struct Receiver_SyncWait
3  {
4      State_SyncWait<T, E>* _state = nullptr;
5      template<typename U>
6      void set_value(U&& v) noexcept
7      {
8          _state->template emplace<1>(FWD(v));
9          _state->_done.set_value();
10     }
11     template<typename U>
12     void set_error(U&& e) noexcept
13     {
14         _state->template emplace<2>(FWD(e));

```

```

15         _state->_done.set_value();
16     }
17     void set_stopped() noexcept
18     {
19         _state->template emplace<0>();
20         _state->_done.set_value();
21     }
22 };

```

It's a generic Receiver that remembers the values (by forwarding to State_SyncWait) and signalling done event. State_SyncWait is:

```

1 template<typename T, typename E>
2 struct State_SyncWait : sync_wait_result<T, E>
3 {
4     std::promise<void> _done;
5     sync_wait_result<T, E>& get()
6     {
7         return *this;
8     }
9     void wait()
10    {
11        _done.get_future().wait();
12    }
13 };

```

which holds sync_wait_result<T, E> that we use to save the values/errors and std::promise<void> which we (ab)use to implement that blocking waiting.

That allows to wait for any Sender:

```

1 int main()
2 {
3     auto r = sync_wait(just(4));
4     assert(r.has_value());
5     assert(r.value() == 4);
6 }

```

Lets continue and implement then() - which allows to attach a lambda to a Sender complete event:

```

1 int main()
2 {
3     auto r = sync_wait(then(just(3)
4         , [](int v) -> void_t
5         {
6             std::println("{} ", v); // prints 3
7             return {};
8         }
9     ));
10 }

```

```

8     });
9     assert(r.has_value());
10 }

```

then() accepts any Sender and invokes a given lambda when that Sender completes. Lambda accepts the result of the Sender and returns a new value. That new value is what a then-sender would return. Basically, then() transforms another Sender's value.

```

1 template<typename Sender, typename Lambda>
2 auto then(Sender&& s, Lambda&& f) noexcept
3 {
4     return Sender_Then<REMOVE_CVR(Sender), REMOVE_CVR(Lambda)>
5         {._s = FWD(s), ._f = FWD(f)};
6 }

```

See, we return Sender_Then that remembers inner sender that we would invoke and a lambda that we would call:

```

1 template<typename Sender, typename Lambda>
2 struct Sender_Then
3 {
4     using inner_value_t = sender_value_t<Sender>;
5     using value_t = std::invoke_result_t<Lambda, inner_value_t>;
6     using error_t = sender_error_t<Sender>;
7     Sender _s;
8     Lambda _f;
9     template<typename Receiver>
10    auto connect(Receiver&& r) noexcept
11    {
12        using Receiver_ = Receiver_Then<REMOVE_CVR(Receiver), Lambda>;
13        return ::connect(MOV(_s), Receiver_{._r = FWD(r), ._f = MOV(_f)});
14    }
15 };

```

Sender_Then:

- leaves error value (error_t) unchanged - the same as the original Sender since we do not touch that
- signals that a value type we would return is the result of invoking a lambda; and
- on connect(), we just return original state because it so happens that our then implementation does not need to store anything extra; and
- everything goes to our then-receiver

Receiver_Then is:

```

1 template<typename Receiver, typename Lambda>
2 struct Receiver_Then

```

```

3  {
4      Receiver _r;
5      Lambda _f;
6      template<typename U>
7      void set_value(U&& v) noexcept
8      {
9          ::set_value(MOV(_r), MOV(_f)(FWD(v)));
10     }
11     template<typename U>
12     void set_error(U&& e) noexcept
13     {
14         ::set_error(MOV(_r), FWD(e));
15     }
16     void set_stopped() noexcept
17     {
18         ::set_stopped(MOV(_r));
19     }
20 };

```

See, `set_value()` just gets a value, calls a lambda and sends that to a next receiver.

That's all. We are forced to return `void_t` since we do not support void values. But other than that, it's a complete implementation:

```

1  sync_wait(then(just(3)
2      , [](int v) -> void_t
3      {
4          std::println("{} ", v); // prints 3
5          return {};
6      }));

```

Finally, to show some async work, let's implement simple `async()` sender that completes the work on thread pool (using `std::async`):

```

1  template<typename Receiver>
2  struct State_Async
3  {
4      Receiver _r;
5      std::future<void> _f;
6      void start() noexcept
7      {
8          _f = std::async(std::launch::async
9              , [r = MOV(_r)]() mutable
10             {
11                 ::set_value(MOV(r), void_t());
12             });

```

```

13     }
14 };
15
16 struct Sender_Async
17 {
18     using value_t = void_t;
19     using error_t = void_t;
20     template<typename Receiver>
21     auto connect(Receiver&& r) noexcept
22     {
23         return State_Async<REMOVE_CVR(Receiver)>{._r = FWD(r)};
24     }
25 };
26
27 auto async()
28 {
29     return Sender_Async{};
30 }

```

`std::async()` there is just for illustrative purpose, has nothing to do with senders and unused everywhere else.

Connecting `async()` with `then()` allows to execute a lambda on a worker thread:

```

1  int main()
2  {
3      const auto main_thread_id = std::this_thread::get_id();
4      std::thread::id work_thread_id;
5      auto x = then(async()
6                  , [&](void_t) -> void_t
7                  {
8                      work_thread_id = std::this_thread::get_id();
9                      return {};
10                 });
11     auto r = sync_wait(MOV(x));
12     assert(r.has_value());
13     assert(main_thread_id != work_thread_id);
14 }

```

9.3 #senders basics: implementing `when_all()`

[source code](#).

Lets implement `when_all()` senders algorithm (sender adaptor, per [p2300r10](#)):

```

1  int main()
2  {

```

```

3     auto op = when_all(
4         just(6)
5         , just('v')
6         , just(7.2)
7     );
8     sync_wait(then(MOV(op), [] (auto vs) -> void_t
9     {
10         auto [a, b, c] = vs;
11         std::println("{} {} {}", a, b, c);
12         return {};
13     }));
14 }

```

Conceptually, we are given a set of N senders and we:

1. start all of them at once;
2. wait for completion of every sender/operation; and
3. finish once everything is done.

Given N senders, we are going to have N values at the end, so we return a tuple of all of the values in a successful case - `std::tuple<int, char, double>` for an example above.

What happens when one of the senders fails? We just return this first error and discard all of the results. Since we store one error value, but there are N error types, we return `std::variant<E1, E2, ...>`.

Similarly, when one of the senders is cancelled and we receive `set_stopped()`, we finish with `set_stopped()` too, discarding/ignoring all of the values.

Real `std::exec` implementation cancels all of the senders yet-in-progress when first error or cancel arrives. For our simplified senders implementation, cancellation is not implemented so we do nothing and simply ensure all of the senders/operations complete (as if cancelled, but none of the senders support cancellation).

Handling variadic set of Senders, each of which could send different types for values and errors is a bit noisy, but lets start with `when_all()`:

```

1 template<typename... Senders>
2 auto when_all(Senders&&... ss)
3 {
4     static_assert(sizeof...(Senders) >= 1);
5     return Sender_When_All<REMOVE_CVR(Senders)...>{FWD(ss)...};
6 }

```

where we accept one or more Senders, construct our Sender wrapper - `Sender_When_All` - which remembers all the Senders, since we need to start them later:

```

1  template<typename... Senders>
2  struct Sender_When_All
3  {
4      using value_t = std::tuple<sender_value_t<Senders>...>;
5      using error_t = std::variant<sender_error_t<Senders>...>;
6
7      std::tuple<Senders...> _ss;
8
9      template<typename... Ss>
10     Sender_When_All(Ss&&... ss)
11         : _ss{FWD(ss)...}
12     {
13     }
14
15     template<typename Receiver>
16     auto connect(Receiver&& r) noexcept
17     {
18         using State = State_When_All<
19             REMOVE_CVR(Receiver)
20             , std::tuple<Senders...>
21             , std::index_sequence_for<Senders...>
22             >;
23         return State{._results{._r = FWD(r)}, ._ss{MOV(_ss)}};
24     }
25 };

```

Sender_When_All does:

- propagate its set_value() type which is a tuple of all of the Senders values
- propagate its set_error() type (a variant of errors)
- remember all of the Senders - to be passed later, on connect()

connect() of our when_all() Sender just returns the (operation) state - State_When_All:

```

1  template<typename Receiver, typename... Senders, auto... Is>
2  struct State_When_All<Receiver
3      , std::tuple<Senders...>          // Senders tuple
4      , std::index_sequence<Is...>    // Senders indexes
5      >
6  {
7      using Results = Result_When_All<
8          Receiver
9          , std::index_sequence<Is...>
10         , Senders...
11         >;
12     using operations_tuple_t = std::tuple<

```

```

13         state_storage_t<Senders
14             , Receiver_When_All<Is, Results>
15             >...
16     >;
17     using senders_tuple = std::tuple<Senders...>;
18
19     Results _results;
20     senders_tuple _ss;
21     operations_tuple_t _states;
22
23     template<auto I>
24     void apply_sender()
25     {
26         using Receiver_ = Receiver_When_All<I, Results>;
27
28         auto& sender = std::get<I>(_ss);
29         auto& state_storage = std::get<I>(_states);
30         auto& state = state_storage.template emplace<I>(
31             ::connect(MOV(sender), Receiver_{._r = &_amp;_results}));
32         ::start(state);
33     }
34
35     void start() noexcept
36     {
37         (apply_sender<Is>(), ...);
38     }
39 };

```

where starting when_all() operation - starts all of the Senders - they could be “executing” concurrently or even in parallel. We have N operations active.

See how our when_all() state embeds all of the other Senders operations states inline - everything is known at compile time:

```

1 template<typename Sender, typename Receiver>
2 using operation_state_t = decltype(::connect(
3     std::declval<Sender>(), std::declval<Receiver>()));
4
5 // To handle non-default-constructible states.
6 template<typename Sender, typename Receiver>
7 using state_storage_t = std::variant<std::monostate
8     , operation_state_t<Sender, Receiver>>;
9
10 using operations_tuple_t = std::tuple<
11     state_storage_t<Senders
12         , Receiver_When_All<Senders, Is, Results>

```

```

13         >...
14     >;
15
16 operations_tuple_t _states;

```

Note, that we need to complete our operation when only last operation ends. To achieve this, we have our own, custom, per-sender Receiver - Receiver_When_All:

```

1 using Receiver_ = Receiver_When_All<I, Results>;
2 auto& state = state_storage.template emplace<I>(
3     ::connect(MOV(sender), Receiver_{._r = &_amp;results}));
4 ::start(state);

```

so when one of the Senders completes, we notify shared results that given operation (indexed by I) is done:

```

1 template<typename Sender, auto I, typename Results>
2 struct Receiver_When_All
3 {
4     Results* _r = nullptr;
5
6     template<typename U>
7     void set_value(U&& v) noexcept
8     {
9         _r->template set_value<I>(FWD(v));
10    }
11    template<typename U>
12    void set_error(U&& e) noexcept
13    {
14        _r->template set_error<I>(FWD(e));
15    }
16    void set_stopped() noexcept
17    {
18        _r->template set_stopped<I>();
19    }
20 };

```

Results must be shared since we count completed operations:

```

1 template<typename Receiver, auto... Is, typename... Senders>
2 struct Result_When_All<Receiver, std::index_sequence<Is...>, Senders...>
3 {
4     using Results = std::tuple<
5         sync_wait_result<
6             sender_value_t<Senders>
7             , sender_error_t<Senders>
8             >...

```

```

9         >;
10     std::mutex _lock;
11     Receiver _r;
12     Results _rs;
13     bool _has_error = false;
14     bool _was_stopped = false;
15     std::int32_t _count = sizeof...(Senders);
16
17     template<auto I, typename U>
18     void set_value(U&& v) noexcept
19     {
20         std::lock_guard _(_lock);
21         if ((_has_error || _was_stopped) == false)
22         {
23             std::get<I>(_rs).set_value(FWD(v));
24         }
25         try_finish();
26     }
27     template<auto I, typename U>
28     void set_error(U&& e) noexcept
29     {
30         std::lock_guard _(_lock);
31         if ((_has_error || _was_stopped) == false)
32         {
33             std::get<I>(_rs).set_error(FWD(e));
34             _has_error = true;
35             // real when_all() - also cancels the rest
36         }
37         try_finish();
38     }
39     template<auto I>
40     void set_stopped() noexcept
41     {
42         std::lock_guard _(_lock);
43         if ((_has_error || _was_stopped) == false)
44         {
45             std::get<I>(_rs).set_stopped();
46             _was_stopped = true;
47             // real when_all() - also cancels the rest
48         }
49         try_finish();
50     }
51     // ...
52 };

```

see, when I(th) operation completes with success, we invoke `set_value<I>()` which remembers the value to final tuple of all of the results. In addition:

- when error or cancel/stop was already done, we skip `set_value`
- for `set_error()`, we remember the error once
- same for a stop.

Note, how `set_error()`, `set_value()` and `set_stopped()` could be invoked all at the same time since we could have potentially truly parallel operations that complete all at once. Since all of them access same, shared state, we do need to have some kind of lock guard in place.

Finally, `try_finish()` decrements operations in progress and completes when last operation completes (`_count == 0`):

```

1 void Result_When_All::try_finish()
2 {
3     _count -= 1;
4     assert(_count >= 0);
5     if (_count == 0)
6     {
7         finish();
8     }
9 }
10
11 void Result_When_All::finish()
12 {
13     if (_was_stopped)
14     {
15         ::set_stopped(MOV(_r));
16     }
17     else if (_has_error)
18     {
19         std::variant<sender_error_t<Senders>...> es;
20         ((std::get<Is>(_rs).has_error()
21          ? (void)es.template emplace<Is>(
22             MOV(std::get<Is>(_rs).error()))
23          : (void)0
24          ), ...);
25         ::set_error(MOV(_r), MOV(es));
26     }
27     else
28     {
29         ::set_value(MOV(_r)
30                     , std::tuple<sender_value_t<Senders>...>(
31                         std::get<Is>(_rs).value()...
32                     )

```

```

33         );
34     }
35 }

```

Handling templates a bit obfuscates the code, but overall:

- if any Sender was stopped, with complete with `set_stopped()`
- if there was an Error, we construct our error variant (with a proper index) and complete with `set_error()`; finally
- when everything completed successfully, we construct a tuple of all of the results and invoke `set_value()` on our target Receiver.

That allows to wait for N operations:

```

1  int main()
2  {
3      auto async_just = [] (auto v)
4      {
5          return then(async(), [copy = MOV(v)] (void_t) mutable
6          {
7              return MOV(copy);
8          });
9      };
10     auto op = when_all(
11         async_just(6)
12         , async_just('v')
13         , just(7.2)
14     );
15     sync_wait(then(MOV(op), [] (auto vs) -> void_t
16     {
17         auto [a, b, c] = vs;
18         std::println("{} {} {}", a, b, c);
19         return {};
20     }));
21 }

```

(See how we composed `async()` and `then()` to create `async_just()` sender that completes on a worker thread).

In addition, the code for this section implements `just_error()` and `just_stopped()` that are identical to `just()`, but complete with `set_error()` and `set_stopped()`.

```

1  int main()
2  {
3      auto r = sync_wait(when_all(
4          just(1)
5          , just_error('x')
6      ));

```

```

7     assert(r.has_error());
8     auto e = r.error();
9     assert(e.index() == 1);
10    assert(std::get<1>(e) == 'x');
11 }

```

9.4 #senders basics: implementing sequence()

[source code](#).

Similar to `when_all()`, given a set of N Senders, we need to start all of them one by one, in order. We simplify and require all of the Senders to return void on success and error:

```

1 template<typename... Senders>
2 auto sequence(Senders&&... ss)
3 {
4     static_assert(sizeof...(Senders) >= 1);
5     static_assert(std::conjunction_v<
6         std::is_same<void_t, sender_value_t<Senders>>...>
7         , "we expect all of the Senders to return void on success");
8     static_assert(std::conjunction_v<
9         std::is_same<void_t, sender_error_t<Senders>>...>
10        , "we expect all of the Senders to return void on error");
11    return Sender_Sequence<REMOVE_CVR(Senders)...>{FWD(ss)...};
12 }

```

`Sender_Sequence` is the same as `Sender_When_All`, we just return `State_Sequence`:

```

1 template<...>
2 struct State_Sequence
3 {
4     Receiver _r;
5     senders_tuple_t _ss;
6     operations_tuple_t _states;
7
8     template<auto I>
9     void apply_sender();
10
11    void start() noexcept
12    {
13        apply_sender<0>();
14    }
15
16    void done()
17    {

```

```

18         ::set_value(MOV(_r), void_t{});
19     }
20 };

```

sequence() operation start() just starts the 1st (at index 0) Sender, by invoking apply_sender<0>(). The logic of chaining - starting the next operation is in apply_sender(), where we start next one (I + 1), when previous completes:

```

1  template<auto I>
2  void State_Sequence::apply_sender()
3  {
4      auto apply_next = [this]()
5      {
6          if constexpr ((I + 1) < sizeof...(Senders))
7          {
8              apply_sender<I + 1>();
9          }
10         else
11         {
12             done();
13         }
14     };
15
16     auto& sender = std::get<I>(_ss);
17     auto& state_storage = std::get<I>(_states);
18     auto& state = state_storage.template emplace<1>(
19         ::connect(MOV(sender)
20             , Receiver_Sequence{._finish = MOV(apply_next)}));
21     ::start(state);
22 }

```

Our internal Receiver_Sequence just invokes completion callback we pass to it, which is apply_next():

```

1  struct Receiver_Sequence
2  {
3      std::function<void ()> _finish; // should not allocate
4      template<typename U>
5      void set_value(U&&) noexcept
6      {
7          _finish();
8      }
9      template<typename U>
10     void set_error(U&&) noexcept
11     {
12         _finish();
13     }

```

```

14     void set_stopped() noexcept
15     {
16         _finish();
17     }
18 };

```

All in all, we can:

```

1  int main()
2  {
3      auto just_log = [](const char* text)
4      {
5          return then(async(), [text](void_t) -> void_t
6          {
7              std::println("{} ", text);
8              return {};
9          });
10     };
11     sync_wait(sequence(just_log("one"), just_log("two")));
12 }

```

9.5 #senders basics: CURL get

Finally, we can implement the same `CURL_sender_get()` we did with `stdexec`, but using our simplified senders and receivers implementation.

10 #reactive streams

11 #SAMPLES

11.1 #synchronous requests

[source code](#), [API section](#).

For completeness, our trivial case - doing 2 GET requests sequentially:

```

1  static void App_Blocking()
2  {
3      const std::string r1 = CURL_get("localhost:5001/file1.txt");
4      const std::string r2 = CURL_get("localhost:5001/file2.txt");
5      std::println("{} ", r1);
6      std::println("{} ", r2);
7  }
8
9  int main()
10 {

```

```

11     App_Blocking();
12 }

```

11.2 #requests with callbacks

[source code](#), [API section](#).

With callbacks API, there are 2 variations:

1. doing 2 sequential requests, one after another: see `App_CallbacksV0()`
2. doing 2 requests concurrently: see `App_CallbacksV1()`

```

1  int main()
2  {
3      App_CallbacksV0(); // sequential
4      App_CallbacksV1(); // concurrent
5  }

```

All the other sections have the same naming convention and the same `main()`.

SEQUENTIAL requests:

```

1  struct App_StateV0 // sequential
2  {
3      CURL_Async _curl_async{};
4      bool _finished = false;
5      std::string _r1;
6      std::string _r2;
7
8      explicit App_StateV0(CURL_Async curl_async) noexcept
9          : _curl_async(curl_async)
10     {
11     }
12     App_StateV0(const App_StateV0&) = delete;
13     ~App_StateV0() noexcept
14     {
15         assert(_finished);
16     }
17
18     void start()
19     {
20         CURL_async_get(_curl_async, "localhost:5001/file1.txt", this
21             , [](void* user_data, std::string response1)
22             {
23                 App_StateV0& state = *static_cast<App_StateV0*>(user_data);
24                 state._r1 = std::move(response1);
25
26                 CURL_async_get(state._curl_async, "localhost:5001/file2.txt", user_data

```

```

27         , [](void* user_data, std::string response2)
28         {
29             App_StateV0& state = *static_cast<App_StateV0*>(user_data);
30             state._r2 = std::move(response2);
31             state._finished = true;
32             state.done();
33         });
34     });
35 }
36
37 void done()
38 {
39     assert(_finished);
40     std::println("{} ", _r1);
41     std::println("{} ", _r2);
42 }
43 };
44
45 static void App_CallbacksV0()
46 {
47     CURL_Async curl_async = CURL_async_create();
48     App_StateV0 app{curl_async};
49     app.start();
50     while (!app._finished)
51     {
52         CURL_async_tick(curl_async);
53     }
54     CURL_async_destroy(curl_async);
55 }

```

CONCURRENT requests:

```

1 struct App_StateV1 // concurrent
2 {
3     CURL_Async _curl_async{};
4     std::int32_t _requests = 0;
5     bool _finished = false;
6     std::string _r1;
7     std::string _r2;
8
9     explicit App_StateV1(CURL_Async curl_async) noexcept
10         : _curl_async(curl_async)
11     {
12     }
13     App_StateV1(const App_StateV1&) = delete;
14     ~App_StateV1() noexcept

```

```

15     {
16         assert(!_finished);
17     }
18
19     void start()
20     {
21         _requests = 2;
22         CURL_async_get(_curl_async, "localhost:5001/file1.txt", this
23             , [](void* user_data, std::string response)
24             {
25                 App_StateV1& state = *static_cast<App_StateV1*>(user_data);
26                 state._r1 = std::move(response);
27                 state.try_finish();
28             });
29         CURL_async_get(_curl_async, "localhost:5001/file2.txt", this
30             , [](void* user_data, std::string response)
31             {
32                 App_StateV1& state = *static_cast<App_StateV1*>(user_data);
33                 state._r2 = std::move(response);
34                 state.try_finish();
35             });
36     }
37
38     void try_finish()
39     {
40         assert(_requests > 0);
41         _requests -= 1;
42         if (_requests == 0)
43         {
44             _finished = true;
45             done();
46         }
47     }
48
49     void done()
50     {
51         assert(_finished);
52         std::println("{} ", _r1);
53         std::println("{} ", _r2);
54     }
55 };
56
57 static void App_CallbacksV1()
58 {
59     CURL_Async curl_async = CURL_async_create();

```

```

60     App_StateV1 app{curl_async};
61     app.start();
62     while (!app._finished)
63     {
64         CURL_async_tick(curl_async);
65     }
66     CURL_async_destroy(curl_async);
67 }

```

11.3 #requests with coroutines

[source code](#), [API section](#).

SEQUENTIAL requests:

```

1  static Co_Task<void> Coro_MainV0(CURL_Async curl_async) // sequential
2  {
3      const std::string r1 = co_await CURL_await_get(curl_async, "localhost:5001/file1.txt");
4      const std::string r2 = co_await CURL_await_get(curl_async, "localhost:5001/file2.txt");
5      std::println("{} ", r1);
6      std::println("{} ", r2);
7  }
8
9  static void App_CoroutinesV0()
10 {
11     CURL_Async curl_async = CURL_async_create();
12     Co_Task<void> task = Coro_MainV0(curl_async);
13     task.resume();
14     while (task.is_in_progress())
15     {
16         CURL_async_tick(curl_async);
17     }
18     CURL_async_destroy(curl_async);
19 }

```

main() loop takes a bit of space, but the actual business logic is almost the same as regular synchronous code.

CONCURRENT requests:

```

1  static Co_Task<void> Coro_MainV1(CURL_Async curl_async) // concurrent
2  {
3      auto [r1, r2] = co_await CO_await_all(
4          CURL_coro_get(curl_async, "localhost:5001/file1.txt"),
5          CURL_coro_get(curl_async, "localhost:5001/file2.txt"));
6      std::println("{} ", r1);
7      std::println("{} ", r2);

```

```

8   }
9
10  static void App_CoroutinesV1()
11  {
12      CURL_Async curl_async = CURL_async_create();
13      Co_Task<void> task = Coro_MainV1(curl_async);
14      task.resume();
15      while (task.is_in_progress())
16      {
17          CURL_async_tick(curl_async);
18      }
19      CURL_async_destroy(curl_async);
20  }

```

11.4 #requests with fibers

[source code](#), [API section](#).

SEQUENTIAL requests:

```

1  static void Fiber_MainV0(CURL_Async curl_async) // sequential
2  {
3      const std::string r1 = CURL_fiber_get(curl_async, "localhost:5001/file1.txt");
4      const std::string r2 = CURL_fiber_get(curl_async, "localhost:5001/file2.txt");
5      std::println("{} ", r1);
6      std::println("{} ", r2);
7  }
8
9  static void App_FibersV0()
10 {
11     Fiber::Boot _;
12     FiberPool fiber_pool{8};
13     FiberTaskScheduler fibers_scheduler{fiber_pool};
14     CURL_Async curl_async = CURL_async_create();
15     FiberTask<void> task = FF_async(fibers_scheduler
16         , &Fiber_MainV0, curl_async);
17     while (task.is_completed() == false)
18     {
19         CURL_async_tick(curl_async);
20         fibers_scheduler.schedule();
21     }
22     CURL_async_destroy(curl_async);
23 }

```

CONCURRENT requests:

```

1  static void Fiber_MainV1( // concurrent
2      FiberTaskScheduler* fiber_scheduler, CURL_Async curl_async)
3  {
4      auto [r1, r2] = FF_await_all(
5          CURL_fiber_get(curl_async, "localhost:5001/file1.txt", *fiber_scheduler),
6          CURL_fiber_get(curl_async, "localhost:5001/file2.txt", *fiber_scheduler)
7      );
8      std::println("{} ", r1);
9      std::println("{} ", r2);
10 }
11
12 static void App_FibersV1()
13 {
14     Fiber::Boot _;
15     FiberPool fiber_pool{8};
16     FiberTaskScheduler fibers_scheduler{fiber_pool};
17     CURL_Async curl_async = CURL_async_create();
18     FiberTask<void> task = FF_async(fibers_scheduler
19         , &Fiber_MainV1, &fibers_scheduler, curl_async);
20     while (task.is_completed() == false)
21     {
22         CURL_async_tick(curl_async);
23         fibers_scheduler.schedule();
24     }
25     CURL_async_destroy(curl_async);
26 }

```

11.5 #polling requests with std::futures

[source code](#), [API section](#).

SEQUENTIAL requests:

```

1  //////////////////////////////////////
2  struct App_StateV0 // sequential
3  {
4      CURL_Async _curl_async{};
5      bool _finished = false;
6      std::optional<std::future<std::string>> _r1;
7      std::optional<std::future<std::string>> _r2;
8
9      explicit App_StateV0(CURL_Async curl_async) noexcept
10         : _curl_async(curl_async)
11     {
12     }
13     App_StateV0(const App_StateV0&) = delete;

```

```

14 ~App_StateV0() noexcept
15 {
16     assert(_finished);
17 }
18
19 void tick()
20 {
21     if (_finished)
22     {
23         return;
24     }
25     if ((_r1.has_value() == false)
26         && (_r2.has_value() == false))
27     { // nothing started yet, run 1st task
28         _r1 = CURL_future_get(_curl_async, "localhost:5001/file1.txt");
29         return;
30     }
31     if (_r1.has_value()
32         && (_r2.has_value() == false))
33     { // 1st task is in progress
34         if (is_future_ready(_r1.value()) == false)
35         {
36             return;
37         }
38         _r2 = CURL_future_get(_curl_async, "localhost:5001/file2.txt");
39         return;
40     }
41     // 2nd task is in progress
42     assert(_r1.has_value() && _r2.has_value());
43     if (is_future_ready(_r2.value()) == false)
44     {
45         return;
46     }
47     _finished = true;
48     done();
49     _r1.reset();
50     _r2.reset();
51 }
52 void done()
53 {
54     assert(_finished);
55     std::println("{} ", _r1->get());
56     std::println("{} ", _r2->get());
57 }
58 };

```

```

59
60 static void App_PollingV0()
61 {
62     CURL_Async curl_async = CURL_async_create();
63     App_StateV0 app{curl_async};
64     while (app._finished == false)
65     {
66         CURL_async_tick(curl_async);
67         app.tick();
68     }
69     CURL_async_destroy(curl_async);
70 }

CONCURRENT requests:

1 struct App_StateV1 // concurrent
2 {
3     CURL_Async _curl_async{};
4     bool _finished = false;
5     std::optional<std::future<std::string>> _r1;
6     std::optional<std::future<std::string>> _r2;
7
8     explicit App_StateV1(CURL_Async curl_async) noexcept
9         : _curl_async(curl_async)
10    {
11    }
12    App_StateV1(const App_StateV0&) = delete;
13    ~App_StateV1() noexcept
14    {
15        assert(!_finished);
16    }
17
18    void tick()
19    {
20        if (_finished)
21        {
22            return;
23        }
24        if ((_r1.has_value() == false)
25            && (_r2.has_value() == false))
26        { // nothing started yet, launch 2 requests
27            _r1 = CURL_future_get(_curl_async, "localhost:5001/file1.txt");
28            _r2 = CURL_future_get(_curl_async, "localhost:5001/file2.txt");
29            return;
30        }
31        assert(_r1.has_value() && _r2.has_value());

```

```

32         if (is_future_ready(_r1.value())
33             && is_future_ready(_r2.value()))
34         {
35             _finished = true;
36             done();
37             _r1.reset();
38             _r2.reset();
39         }
40     }
41     void done()
42     {
43         assert(_finished);
44         std::println("{} ", _r1->get());
45         std::println("{} ", _r2->get());
46     }
47 };
48
49 static void App_PollingV1()
50 {
51     CURL_Async curl_async = CURL_async_create();
52     App_StateV1 app{curl_async};
53     while (app._finished == false)
54     {
55         CURL_async_tick(curl_async);
56         app.tick();
57     }
58     CURL_async_destroy(curl_async);
59 }

```

11.6 #requests with Tasks .then()

[source code](#), [API section](#).

SEQUENTIAL requests:

```

1  static Task<void> Task_MainV0(CURL_Async curl_async) // sequential
2  {
3      return CURL_task_get(curl_async, "localhost:5001/file1.txt")
4          .then([curl_async](std::string r1)
5          {
6              std::println("{} ", r1);
7              return CURL_task_get(curl_async, "localhost:5001/file2.txt")
8                  .then([](std::string r2)
9                  {
10                     std::println("{} ", r2);
11                 });

```

```

12     });
13 }
14
15 static void App_TasksV0()
16 {
17     CURL_Async curl_async = CURL_async_create();
18     Task<void> task = Task_MainV0(curl_async);
19     while (task.has_value() == false)
20     {
21         CURL_async_tick(curl_async);
22     }
23     CURL_async_destroy(curl_async);
24 }

```

CONCURRENT requests:

```

1 static Task<void> Task_MainV1(CURL_Async curl_async) // concurrent
2 {
3     return Tasks_WhenAll(
4         CURL_task_get(curl_async, "localhost:5001/file1.txt"),
5         CURL_task_get(curl_async, "localhost:5001/file2.txt")
6     )
7     .then([](std::tuple<std::string, std::string> vs)
8     {
9         const auto& [r1, r2] = vs;
10        std::println("{} ", r1);
11        std::println("{} ", r2);
12    });
13 }
14
15 static void App_TasksV1()
16 {
17     CURL_Async curl_async = CURL_async_create();
18     Task<void> task = Task_MainV1(curl_async);
19     while (task.has_value() == false)
20     {
21         CURL_async_tick(curl_async);
22     }
23     CURL_async_destroy(curl_async);
24 }

```

11.7 #requests with senders/std::execution

[source code](#), [API section](#).

SEQUENTIAL requests:

```

1 auto Sender_MainV0(CURL_Async curl_async) // sequential
2 {
3     return exec::sequence(
4         CURL_sender_get(curl_async, "localhost:5001/file1.txt")
5         | stdexec::then([](std::string r1)
6         {
7             std::println("{} ", r1);
8         }),
9         CURL_sender_get(curl_async, "localhost:5001/file2.txt")
10        | stdexec::then([](std::string r2)
11        {
12            std::println("{} ", r2);
13        }));
14 }
15
16 void App_SendersV0()
17 {
18     CURL_Async curl_async = CURL_async_create();
19     bool done = false;
20     auto state = stdexec::connect(Sender_MainV0(curl_async), AnyReceiver{&done});
21     stdexec::start(state);
22     while (done == false)
23     {
24         CURL_async_tick(curl_async);
25     }
26     CURL_async_destroy(curl_async);
27 }

```

CONCURRENT requests:

```

1 auto Sender_MainV1(CURL_Async curl_async) // concurrent
2 {
3     return stdexec::when_all(
4         CURL_sender_get(curl_async, "localhost:5001/file1.txt"),
5         CURL_sender_get(curl_async, "localhost:5001/file2.txt"))
6         | stdexec::then([](std::string&& r1, std::string&& r2)
7         {
8             std::println("{} ", r1);
9             std::println("{} ", r2);
10        });
11 }
12
13 void App_SendersV1()
14 {
15     CURL_Async curl_async = CURL_async_create();
16     bool done = false;

```

```
17     auto state = stdexec::connect(Sender_MainV1(curl_async), AnyReceiver{&done});
18     stdexec::start(state);
19     while (done == false)
20     {
21         CURL_async_tick(curl_async);
22     }
23     CURL_async_destroy(curl_async);
24 }
```